

DEPARTMENT OF CSE (IOT & CYBER SECURITY INCLUDING BLOCK CHIAN TECHNOLOGY)

DEVOPS LABORATORY

BCSL657D - 6TH SEMESTER B.E.
Academic Year – 2024-25

LAB MANUAL

Prepared by:
Gayithri N
Assistant Professor
Dept. of CSE

Under the Guidance of:
DR. SAVITHA CHOUDARY
Prof. and Head
Dept. of IOT

Department Vision and Mission

VISION

Building a Proficient Internet of Things Engineers with core values to solve **real time** problems.

MISSION

M1: To offer state-of-the-art Internet of Things education imparting skills for building Cutting-edge and innovative applications with block chain and cyber security.

M2: To inculcate ethically and socially committed Internet of Things professionals through Value-added courses.

M3: Promoting research blend among students and enabling continuous learning

1. Course Description:

This course covers to Learn the syntax and semantics of the Python programming language. Demonstrate the use of built-in functions to navigate the file system

2. Course Objectives:

CLO 1. Learn the syntax and semantics of the Python programming language.

CLO 2. Illustrate the process of structuring the data using lists, tuples.

CLO 3. Appraise the need for working with various documents like Excel, PDF, Word and Others.

CLO 4. Demonstrate the use of built-in functions to navigate the file system

CLO 5. Implement the Object Oriented Programming concepts in Python.

3.Course Outcomes (CO):

CO1	Demonstrate proficiency in handling loops and creation of functions..
CO2	Identify the methods to create and manipulate lists, tuples and dictionaries.
CO3	Develop programs for string processing and file organization
CO4	Interpret the concepts of Object-Oriented Programming as used in Python.

Program Outcomes (PO)		Level	Proficiency assessed by
PO1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.	2	Assignments, Tutorials, Mock Tests
PO2	Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.	2	Assignments, Tutorials
PO3	Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.	3	Assignments, Tutorials, Mock Tests
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.	3	Assignments, Tutorials, Mock Tests
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.	2	Assignments, Tutorials, Mock Tests
PO6	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.	3	Assignments, Tutorials, Mock Tests
PO7	Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.	-	-
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.	-	-
PO9	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.	-	-
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear	-	-
PO11	Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.	2	Assignments, Tutorials, Mock Tests
PO12	Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.	-	-

DEVOPS		Semester	6
Course Code	BCSL657D	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	0:0:2:0	SEE Marks	50
Credits	01	Exam Hours	100
Examination type (SEE)	Practical		
Course objectives: • To introduce DevOps terminology, definition & concepts • To understand the different Version control tools like Git, Mercurial • To understand the concepts of Continuous Integration/ Continuous Testing/ Continuous Deployment) • To understand Configuration management using Ansible • Illustrate the benefits and drive the adoption of cloud-based Devops tools to solve real world problems			
Sl.NO	Experiments		
1	Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup		
2	Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins		
3	Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation		
4	Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle		
5	Introduction to Jenkins: What is Jenkins?, Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use		
6	Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests		
7	Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook		
8	Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins		
9	Introduction to Azure DevOps: Overview of Azure DevOps Services, Setting Up an Azure DevOps Account and Project		
10	Creating Build Pipelines: Building a Maven/Gradle Project with Azure Pipelines, Integrating Code Repositories (e.g., GitHub, Azure Repos), Running Unit Tests and Generating Reports		
11	Creating Release Pipelines: Deploying Applications to Azure App Services, Managing Secrets and Configuration with Azure Key Vault, Hands-On: Continuous Deployment with Azure Pipelines		

12	Practical Exercise and Wrap-Up: Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A
Course outcomes (Course Skill Set): At the end of the course the student will be able to: • Demonstrate different actions performed through Version control tools like Git. • Perform Continuous Integration and Continuous Testing and Continuous Deployment using Jenkins by building and automating test cases using Maven & Gradle. • Experiment with configuration management using Ansible. • Demonstrate Cloud-based DevOps tools using Azure DevOps.	

Introduction to DevOps :

DevOps is a software development methodology that combines software development (Dev) and IT operations (Ops) to streamline the software delivery process. It focuses on automation, collaboration, continuous integration (CI), continuous delivery (CD), and monitoring to ensure faster, more efficient, and reliable software deployment.

In DevOps, teams use build automation tools like Maven and Gradle to automate the compilation, testing, and packaging of applications. These tools help developers manage dependencies, ensure consistent builds, and integrate code changes seamlessly, which are essential for continuous integration and continuous delivery (CI/CD) pipelines.

Key Aspects of DevOps:

- **Collaboration and Communication** – DevOps bridges the gap between development and operations teams, fostering a culture of shared responsibility.
- **Automation** – Automating software builds, testing, deployment, and monitoring reduces manual errors and improves efficiency.
- **Continuous Integration & Continuous Delivery (CI/CD)** – Code changes are integrated frequently, tested automatically, and deployed seamlessly.
- **Infrastructure as Code (IaC)** – Infrastructure setup and management are automated using tools like Terraform and Ansible.
- **Monitoring and Feedback** – Continuous monitoring helps in identifying issues early and improving system performance.

DevOps Lifecycle:

The DevOps lifecycle consists of the following stages:

- ❖ **Plan** – Define project scope, requirements, and workflows using tools like Jira, Confluence.
- ❖ **Develop** – Code development using Git, GitHub, GitLab.
- ❖ **Build** – Compile and package code using Maven, Gradle.
- ❖ **Test** – Automated testing with tools like JUnit, Selenium.
- ❖ **Release** – Prepare for deployment using Jenkins, GitHub Actions.
- ❖ **Deploy** – Deploy applications using Ansible, Docker, Kubernetes.
- ❖ **Operate** – Manage infrastructure and monitor applications using Terraform, Prometheus.
- ❖ **Monitor** – Track system performance and logs using Grafana, ELK Stack.

Benefits of DevOps:

-  **Faster Development & Deployment** – Reduces time from development to production.
-  **Improved Collaboration** – Developers and IT teams work together efficiently.
-  **Better Software Quality** – Automated testing ensures fewer bugs.
-  **Scalability & Reliability** – Applications are more stable and scalable.
-  **Enhanced Security** – Automated security checks prevent vulnerabilities.

1. Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup

Introduction to Maven and Gradle

Overview of Build Automation Tools

Build automation tools are essential in software development for managing dependencies, compiling source code, running tests, and packaging applications. They help in streamlining the development process and ensure reproducibility.

Two popular automation tools in the Java ecosystem are Maven and Gradle. Both are great for managing project builds and dependencies, but they have some key differences.

Maven :

Maven is a build automation tool primarily used for Java projects. It uses an XML configuration file called pom.xml (Project Object Model) to define project settings, dependencies, and build steps.

Main Features:

- Predefined project structure and lifecycle phases.
- Automatic dependency management through Maven Central.
- Wide range of plugins for things like testing and deployment.
- Supports complex projects with multiple modules.

Gradle :

Gradle is a more modern and versatile build tool that supports multiple programming languages, including Java, Groovy, and Kotlin. It uses a domain-specific language (DSL) for build scripts, written in Groovy or Kotlin.

Main Features:

- Faster builds thanks to task caching and incremental builds.
- Flexible and customizable build scripts.
- Works with Maven repositories for dependency management.
- Excellent support for multi-module and cross-language projects.
- Integrates easily with CI/CD pipelines.

Key Differences Between Maven and Gradle

Aspect	Maven	Gradle
Configuration	XML (pom.xml)	Groovy or Kotlin DSL
Performance	Slower	Faster due to caching
Flexibility	Less flexible	Highly customizable
Learning Curve	Easier to pick up	Slightly steeper
Script Size	Verbose	More concise
Dependency Management	Uses Maven Central	Compatible with Maven too
Plugin Support	Large ecosystem	Extensible and versatile

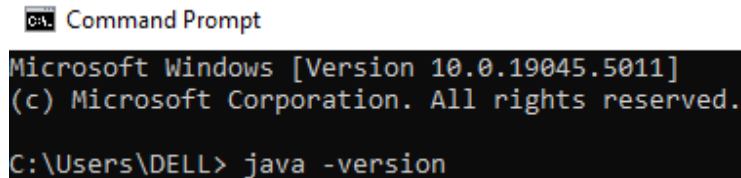
- Maven is great for structured, enterprise-level projects.
- Gradle is ideal for scalable and performance-driven applications.

Installation and Environment Setup

Installing Maven :

1. Install Java (JDK 17 Recommended)

- Check if Java is installed:



```
Command Prompt
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL> java -version
```

2. Download Maven

- Go to the [Maven Download Page](#) and download the latest binary ZIP file.

Binary tar.gz archive	apache-maven-3.8.5-bin.tar.gz	apache-maven-3.8.5-bin.tar.gz.sha512	apache-maven-3.8.5-bin.tar.gz.asc
Binary zip archive	apache-maven-3.8.5-bin.zip	apache-maven-3.8.5-bin.zip.sha512	apache-maven-3.8.5-bin.zip.asc
Source tar.gz archive	apache-maven-3.8.5-src.tar.gz	apache-maven-3.8.5-src.tar.gz.sha512	apache-maven-3.8.5-src.tar.gz.asc
Source zip archive	apache-maven-3.8.5-src.zip	apache-maven-3.8.5-src.zip.sha512	apache-maven-3.8.5-src.zip.asc

3. Extract the ZIP File:

- Right-click the downloaded ZIP file and select **Extract All...** or use any extraction tool like WinRAR or 7-Zip.

4. Move the Folder:

- After extraction, move the extracted **Maven folder** (usually named **apache-maven-x.x.x**) to a convenient directory like **C:\Program Files**.

5. Navigate to the bin Folder:

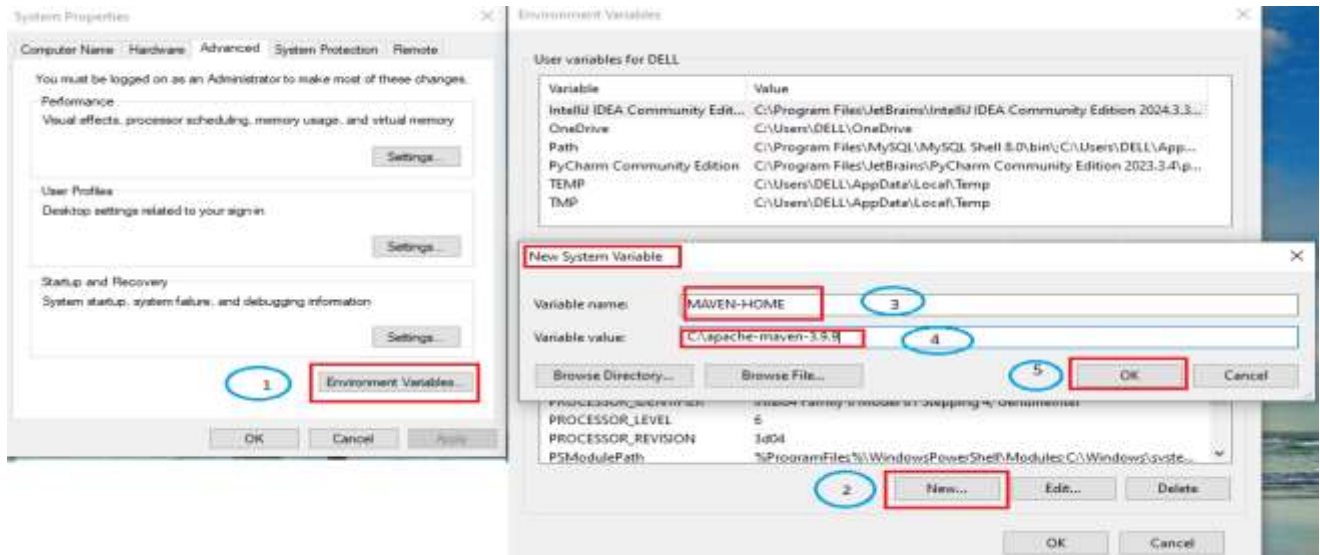
- Open the **Maven folder**, then navigate to the **bin** folder inside.
- Copy the path from the File Explorer address bar (e.g., **C:\Program Files\apache-maven-x.x.x\bin**).

6. Set Environment Variables:

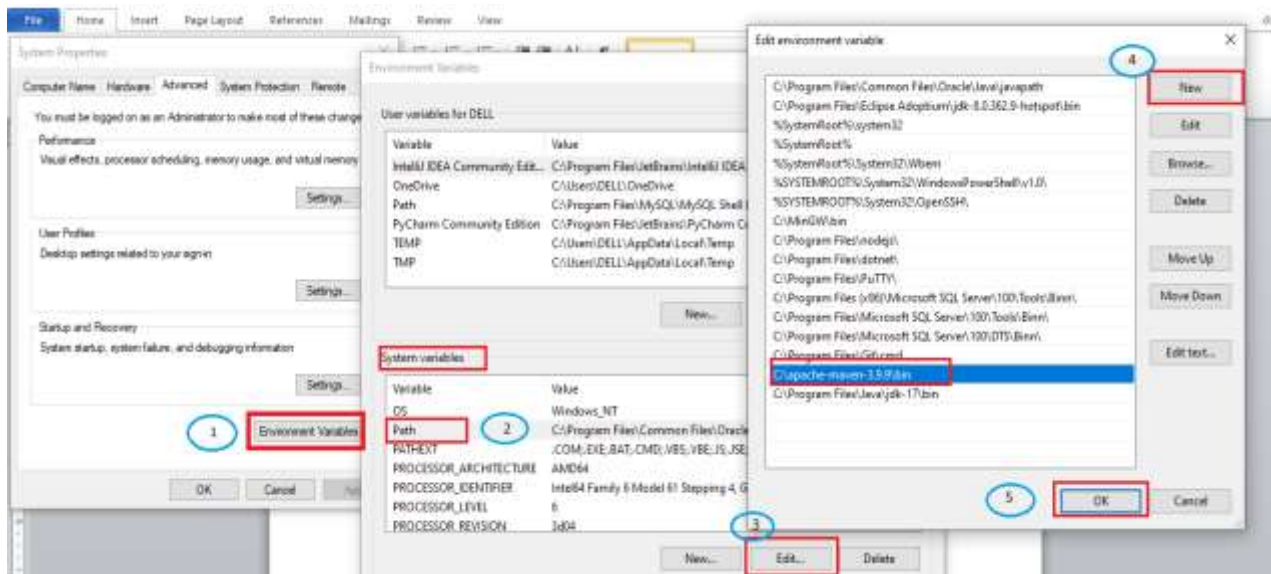
- Open the **Start Menu**, search for **Environment Variables**, and select **Edit the system environment variables**.

- Click **Environment Variables**.
- Under **System Variables**:

(a) click **New** set variable name as MAVEN – HOME , paste the path



(b) Editing “Path” environment variable and adding the maven directory path up until “bin” folder.



7. Verify the Installation:

Open Command Prompt and run: `mvn -version` If Maven is correctly installed, it will display the version number.

```
Command Prompt
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

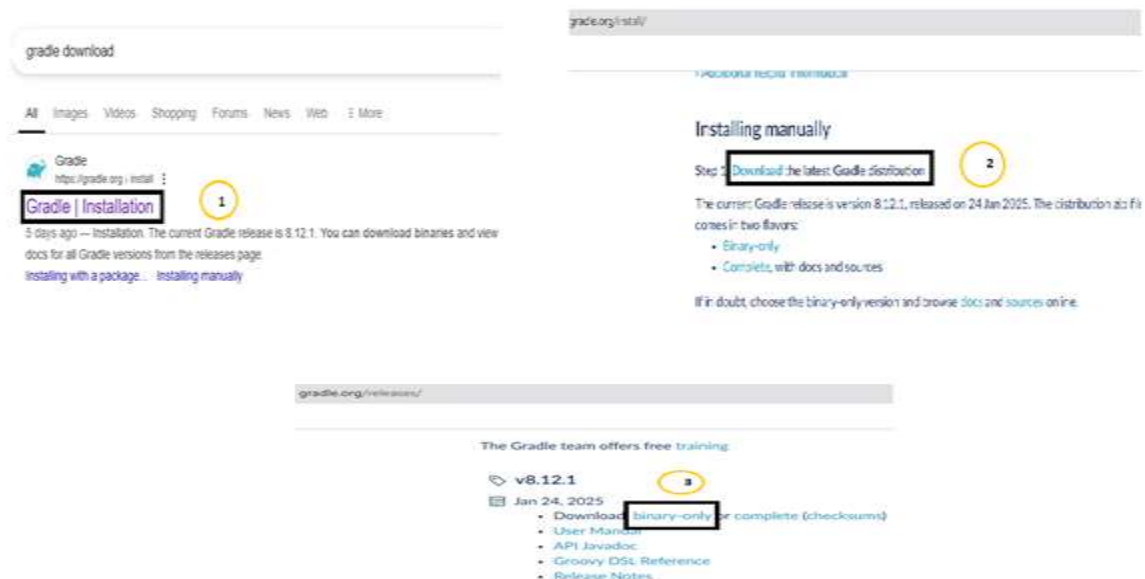
C:\Users\DELL>mvn -version
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfc9d97d260186937)
Maven home: C:\apache-maven-3.9.9
Java version: 17.0.12, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-17
Default locale: en_US, platform encoding: Cp1252
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"

C:\Users\DELL>
```

Installing Gradle

1. Download Gradle :

- Go to the [Gradle Download Page](https://gradle.org/install/) and download the latest binary ZIP file.



2. Extract the ZIP File:

- Right-click the downloaded ZIP file and select **Extract All...** or use any extraction tool like WinRAR or 7-Zip.

3. Move the Folder:

- After extraction, move the extracted **Gradle folder** (usually named **gradle-x.x.x**) to a convenient directory like **C:\Program Files**.

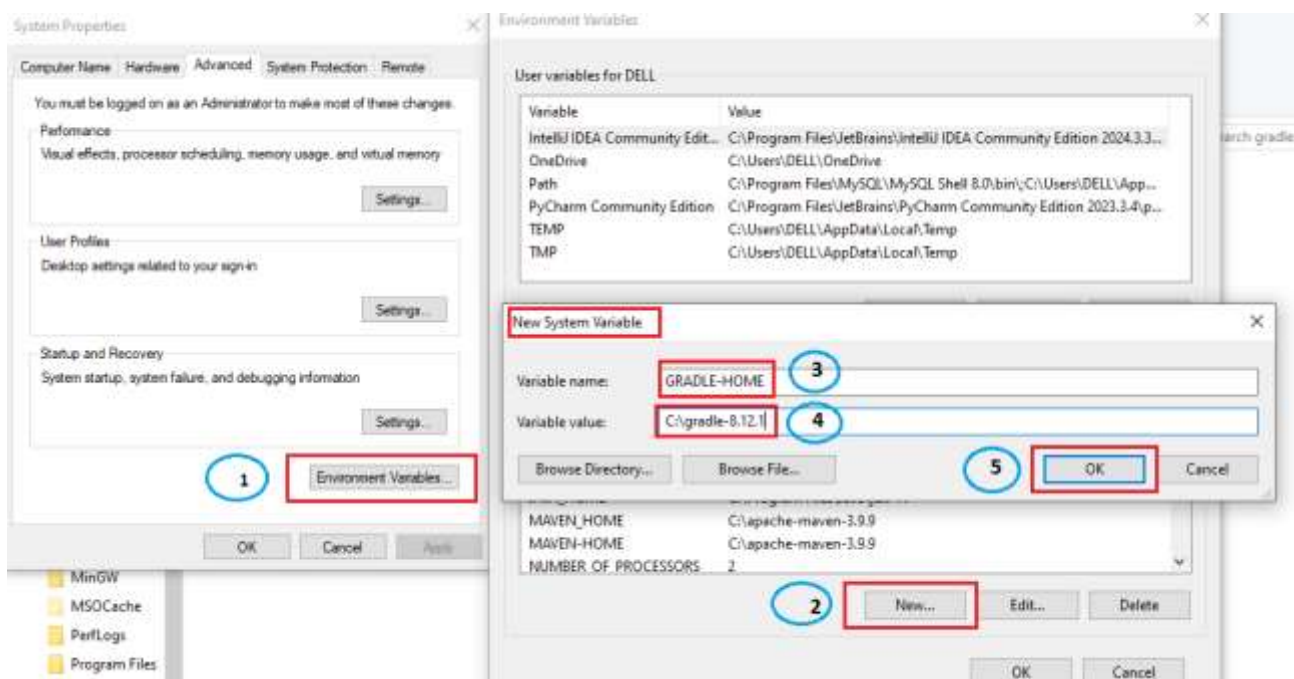
4. Navigate to the bin Folder:

- Open the **Gradle folder**, then navigate to the **bin** folder inside.
- Copy the path from the File Explorer address bar (e.g., **C:\Program Files\gradle-x.x\bin**).

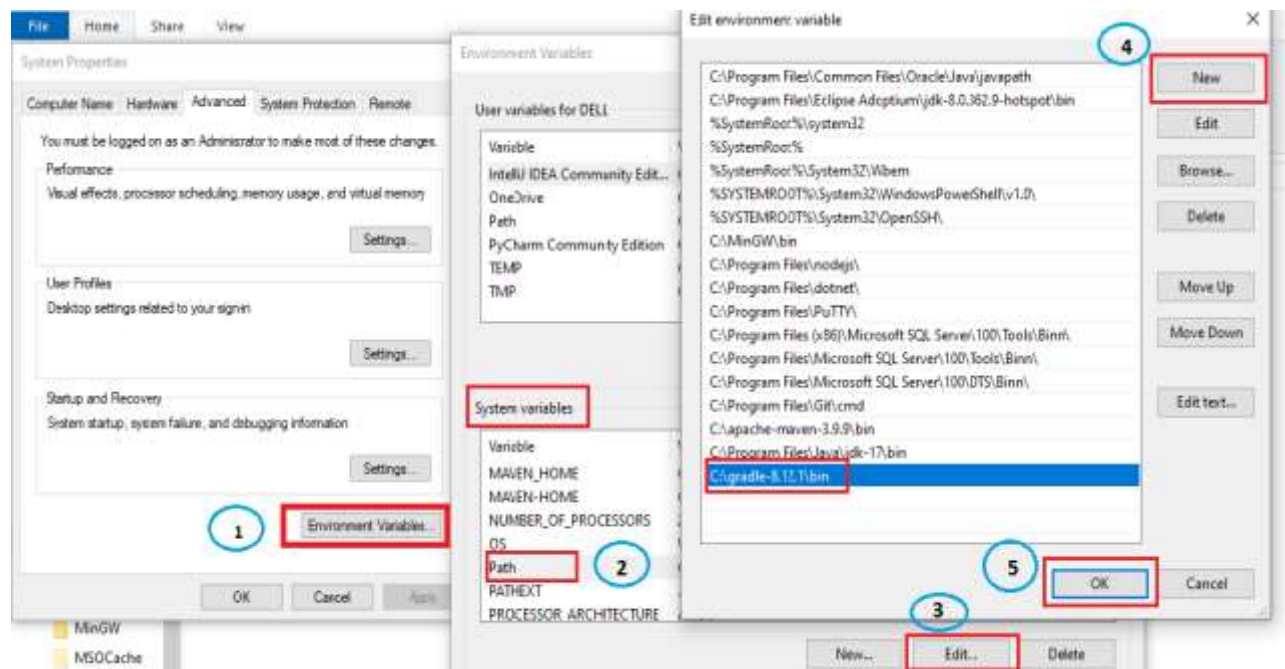
5. Set Environment Variables:

- Open the **Start Menu**, search for **Environment Variables**, and select **Edit the system environment variables**.
- Click **Environment Variables**.
- Under **System Variables**

(a) click **New** set variable name as **GRADLE – HOME** , paste the path



(b) Editing “Path” environment variable and adding the gradle directory path up until “bin” folder.



6. Verify the Installation:

- Open a terminal or Command Prompt and run: **gradle -version** If it shows the Gradle version, the setup is complete.

```

C:\Users\DELL>gradle -version

Welcome to Gradle 8.12.1!

Here are the highlights of this release:
- Enhanced error and warning reporting with the Problems API
- File-system watching support on Alpine Linux
- Build and test Swift 6 libraries and apps

For more details see https://docs.gradle.org/8.12.1/release-notes.html

-----
Gradle 8.12.1
-----
Build time: 2025-01-24 12:55:12 UTC
Revision: 0b1ee1ff81d1f4a26574ff4a362ac9180852b140

Kotlin: 2.0.21
Groovy: 3.0.22
Ant: Apache Ant(TM) version 1.10.15 compiled on August 25 2024
Launcher JVM: 17.0.12 (Oracle Corporation 17.0.12+8-LTS-286)
Daemon JVM: C:\Program Files\Java\jdk-17 (no JDK specified, using current Java home)
OS: Windows 10 10.0 amd64

C:\Users\DELL>

```


2. Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

Overview of the Project

1. Creating a Maven Project

There are a few ways to create a Maven project, such as using the command line, IDEs like IntelliJ IDEA or Eclipse, or generating it via an archetype.

(a) Using Command Line:

- To create a basic Maven project using the command line, you can use the following command

```
E:\Maven-project1>mvn archetype:generate -DgroupId=com.mvit -DartifactId=flipkart-app -DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.4 -DinteractiveMode=false
```

- **groupId**: A unique identifier for the group (usually the domain name).
- **artifactId**: A unique name for the project artifact (your project).
- **archetypeArtifactId**: The template you want to use for the project.
- **DinteractiveMode=false**: Disables prompts during project generation.

This will create a basic Maven project with the required directory structure and pom.xml file.

(b) Using IDEs

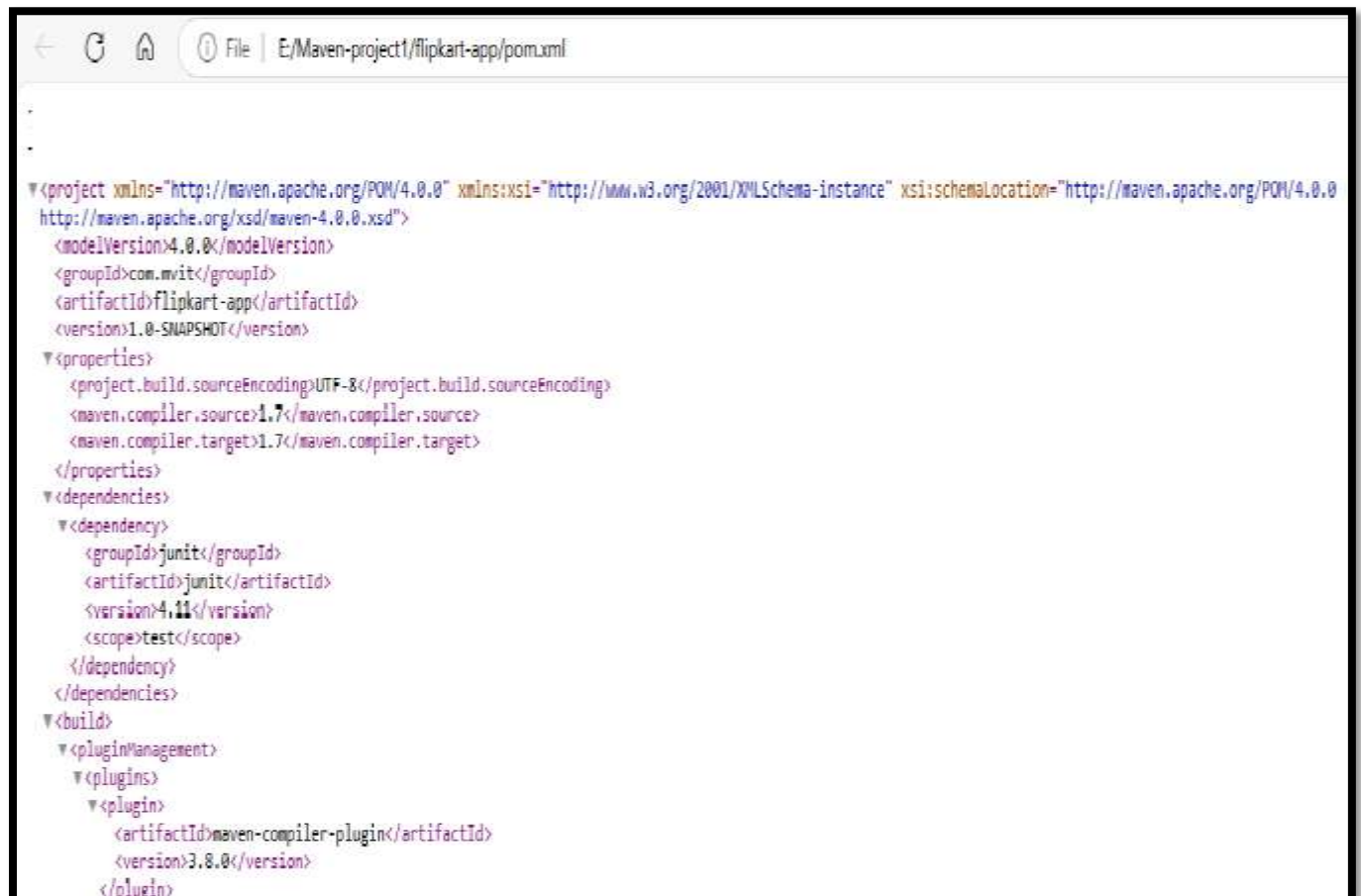
Most modern IDEs (like IntelliJ IDEA or Eclipse) provide wizards to generate Maven projects. For example, in IntelliJ IDEA:

1. Go to **File > New Project**.
2. Choose **Maven** from the list of project types.
3. Provide the **groupId** and **artifactId** for your project.

2. Understanding the POM File

The POM (Project Object Model) file is the heart of a Maven project. It is an XML file that contains all the configuration details about the project.

Below is an example of a simple POM file:



Key element in pom.xml:

- **<groupId>**: The group or organization that the project belongs to.
- **<artifactId>**: The name of the project or artifact.
- **<version>**: The version of the project (often follows a format like 1.0-SNAPSHOT).
- **<packaging>**: Type of artifact, e.g., jar, war, pom, etc.
- **<dependencies>**: A list of dependencies the project requires.
- **<build>**: Specifies the build settings, such as plugins to use.

3. Dependency Management

Maven uses the <dependencies> tag in the pom.xml to manage external libraries or dependencies that your project needs. When Maven builds the project, it will automatically download these dependencies from a repository (like Maven Central).

Example of adding a dependency:

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

- **Transitive Dependencies**
 - Maven automatically resolves transitive dependencies. For example, if you add a library that depends on other libraries, Maven will also download those.
- **Scopes**
 - Dependencies can have different scopes that determine when they are available:
 - **compile** (default): Available in all build phases.
 - **provided**: Available during compilation but not at runtime (e.g., a web server container).
 - **runtime**: Needed only at runtime, not during compilation.
 - **test**: Required only for testing.

4. Using Plugins

Maven plugins are used to perform tasks during the build lifecycle, such as compiling code, running tests, packaging, and deploying. You can specify plugins within the <build> section of your pom.xml.

- **Adding Plugins**
 - You can add a plugin to your pom.xml like so:

```
<build>
  <pluginManagement>
    <plugins>
      <plugin>
        <artifactId>maven-clean-plugin</artifactId>
        <version>3.1.0</version>
      </plugin>
      <plugin>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.0</version>
      </plugin>
      <plugin>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>2.22.1</version>
      </plugin>
    </plugins>
  </pluginManagement>
</build>
```

1. Common Plugins

- **maven-compiler-plugin:** Compiles Java code.
- **maven-surefire-plugin:** Runs unit tests.
- **maven-jar-plugin:** Packages the project as a JAR file.
- **maven-clean-plugin:** Cleans up the **target/** directory.

2. Plugin Goals

Each plugin consists of goals, which are specific tasks to be executed. For example:

- **mvn clean install:** This will clean the target directory and then install the package in the local repository.
- **mvn compile:** This will compile the source code.
- **mvn test:** This will run unit tests.

Working with Maven Project

Note: Always create separate folder to do any program.

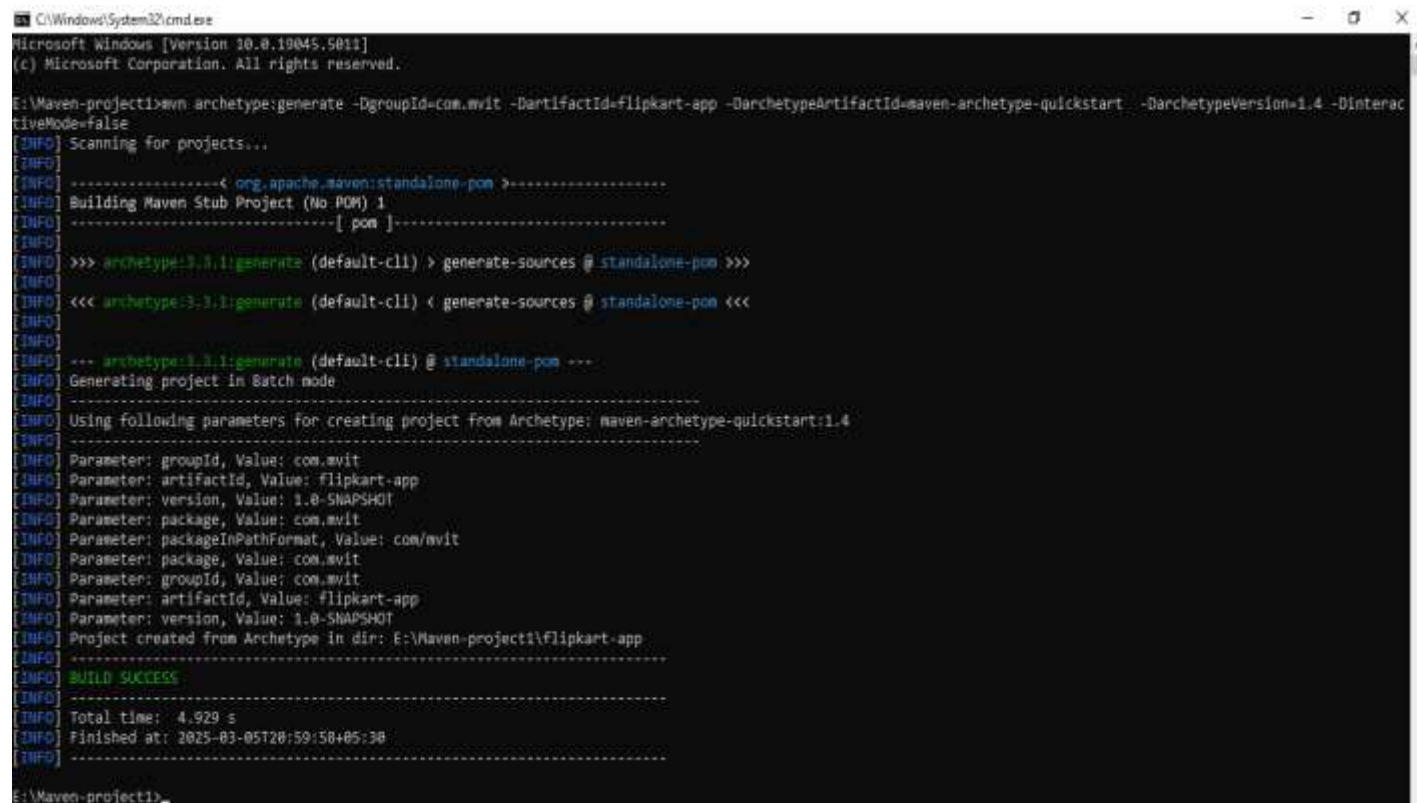
- Create a folder in a local drive D or E and then open a command prompt
- After then follow the below steps to working with Maven project.

Step 1: Creating a Basic Maven Java Application Project

- You can create a Maven project using the mvn command or through an IDE (such as IntelliJ IDEA or Eclipse). Below is the essential pom.xml configuration and corresponding Java code required for the project.
- To create a Maven project using the command line, use the following command:

```
E:\Maven-project1>mvn archetype:generate -DgroupId=com.mvit -DartifactId=flipkart-app -DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.4 -DinteractiveMode=false
```

This command generates a basic maven java application with a predefined structure using the Quickstart archetype.

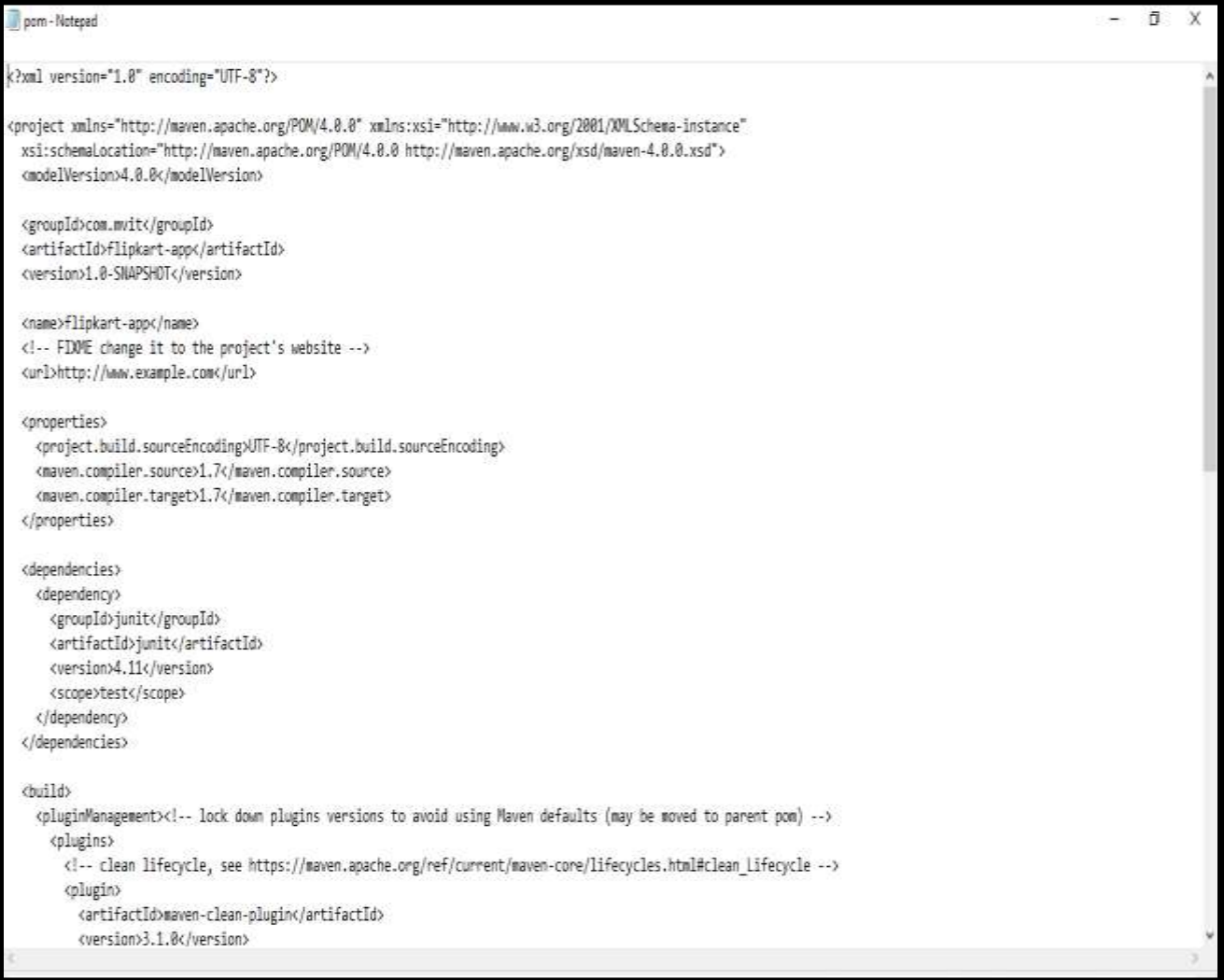


```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

E:\Maven-project1>mvn archetype:generate -DgroupId=com.mvit -DartifactId=flipkart-app -DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.4 -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO] >>> archetype:3.3.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< archetype:3.3.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO] --- archetype:3.3.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] -----
[INFO] Using following parameters for creating project from Archetype: maven-archetype-quickstart:1.4
[INFO] -----
[INFO] Parameter: groupId, Value: com.mvit
[INFO] Parameter: artifactId, Value: flipkart-app
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] Parameter: package, Value: com.mvit
[INFO] Parameter: packageInPathFormat, Value: com/mvit
[INFO] Parameter: package, Value: com.mvit
[INFO] Parameter: groupId, Value: com.mvit
[INFO] Parameter: artifactId, Value: flipkart-app
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] Project created from Archetype in dir: E:\Maven-project1\flipkart-app
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.929 s
[INFO] Finished at: 2025-03-05T20:59:50+05:30
[INFO] -----
E:\Maven-project1>
```

Step 2: Open The pom.xml File

- You can manually navigate the **project folder** named call flipkart-app and open the file pom.xml
- In case if you not getting project folder then type command in your cmd.
 - **cd flipkart-app** – is use to navigate the project folder.
 - **notepad pom.xml** – is use to open pom file in notepad.



```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mvit</groupId>
  <artifactId>flipkart-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <name>flipkart-app</name>
  <!-- FIXME change it to the project's website -->
  <url>http://www.example.com</url>

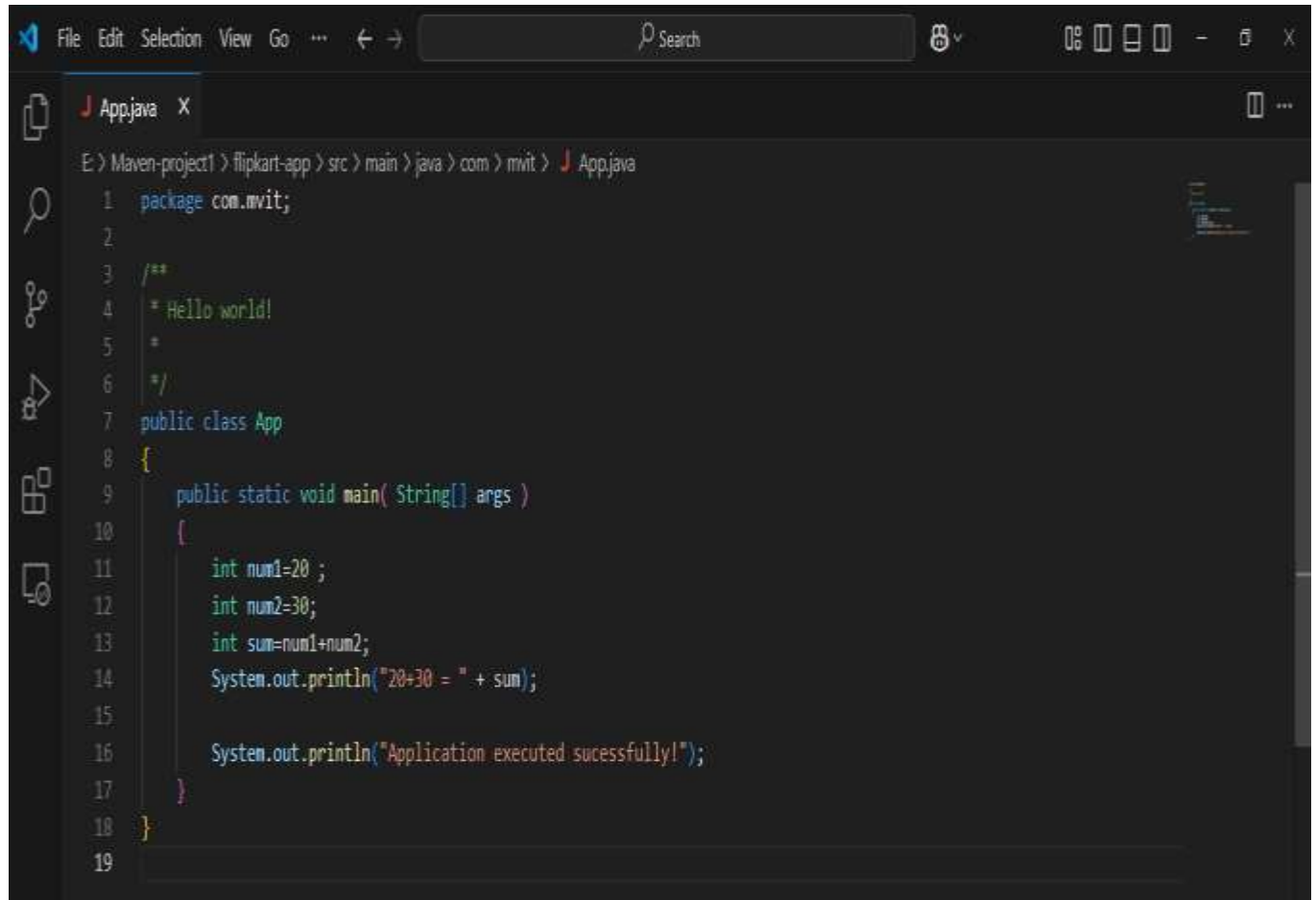
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>1.7</maven.compiler.source>
    <maven.compiler.target>1.7</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <pluginManagement><!-- lock down plugins versions to avoid using Maven defaults (may be moved to parent pom) -->
      <plugins>
        <!-- clean lifecycle, see https://maven.apache.org/ref/current/maven-core/lifecycles.html#clean_lifecycle -->
        <plugin>
          <artifactId>maven-clean-plugin</artifactId>
          <version>3.1.0</version>
```

Step 3: Open Java Code (App.java) File

- Open a file **App.java** inside the **src/main/java/com/mvit/** directory.
- After opening the **App.java** write the simple java program



```
E > Maven-project1 > flipkart-app > src > main > java > com > mvit > J App.java
1 package com.mvit;
2
3 /**
4  * Hello world!
5  *
6  */
7 public class App
8 {
9     public static void main( String[] args )
10    {
11        int num1=20 ;
12        int num2=30;
13        int sum=num1+num2;
14        System.out.println("20+30 = " + sum);
15
16        System.out.println("Application executed sucessfully!");
17    }
18 }
19
```

Note: before building the project make sure you are in the project folder if not navigate the project folder type command in your command prompt **cd flipkart-app**

Step 4: Building the Project

To build and run this project, follow these steps:

1. Compile the Project

C:\Windows\System32\cmd.exe

```
E:\Maven-project1>cd flipkart-app
```

```
E:\Maven-project1\flipkart-app> mvn compile
```

```
C:\Windows\System32\cmd.exe
E:\Maven-project1>cd flipkart-app
E:\Maven-project1\flipkart-app>mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.mvit:flipkart-app >-----
[INFO] Building flipkart-app 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.0.2:resources (default-resources) @ flipkart-app ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory E:\Maven-project1\flipkart-app\src\main\resources
[INFO]
[INFO] --- compiler:3.8.0:compile (default-compile) @ flipkart-app ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to E:\Maven-project1\flipkart-app\target\classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 4.752 s
[INFO] Finished at: 2025-03-05T22:17:41+05:30
[INFO]
E:\Maven-project1\flipkart-app>
```

2. Run the Unit Tests

C:\Windows\System32\cmd.exe

```
E:\Maven-project1\flipkart-app> mvn test
```

```
C:\Windows\System32\cmd.exe
E:\Maven-project1\flipkart-app> mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.mvit:flipkart-app >-----
[INFO] Building flipkart-app 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.0.2/resources (default-resources) @ flipkart-app ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory E:\Maven-project1\flipkart-app\src\main\resources
[INFO]
[INFO] --- compiler:3.8.0/compiler (default-compile) @ flipkart-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.0.2/testResources (default-testResources) @ flipkart-app ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory E:\Maven-project1\flipkart-app\src\test\resources
[INFO]
[INFO] --- compiler:3.8.0:testCompile (default-testCompile) @ flipkart-app ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to E:\Maven-project1\flipkart-app\target\test-classes
[INFO]
[INFO] --- surefire:2.22.1:test (default-test) @ flipkart-app ---
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.mvit.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.169 s - in com.mvit.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 10.522 s
[INFO] Finished at: 2025-03-05T22:29:36+05:30
[INFO]
[INFO] -----
E:\Maven-project1\flipkart-app>
```

3. Package the project into a JAR

C:\Windows\System32\cmd.exe

```
E:\Maven-project1\flipkart-app> mvn package
```

C:\Windows\System32\cmd.exe

```
E:\Maven-project1\flipkart-app>mvn package
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.mvit:flipkart-app >-----
[INFO] Building flipkart-app 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.0.2:resources (default-resources) @ flipkart-app ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory E:\Maven-project1\flipkart-app\src\main\resources
[INFO]
[INFO] --- compiler:3.8.0:compile (default-compile) @ flipkart-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.0.2:testResources (default-testResources) @ flipkart-app ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory E:\Maven-project1\flipkart-app\src\test\resources
[INFO]
[INFO] --- compiler:3.8.0:testCompile (default-testCompile) @ flipkart-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:2.22.1:test (default-test) @ flipkart-app ---
[INFO]
[INFO] T E S T S
[INFO]
[INFO] Running com.mvit.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.352 s - in com.mvit.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.0.2:jar (default-jar) @ flipkart-app ---
[INFO] Building jar: E:\Maven-project1\flipkart-app\target\flipkart-app-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.391 s
[INFO] Finished at: 2025-03-05T22:39:37+05:30
[INFO]
```

4. Run the application (using JAR)

```
C:\Windows\System32\cmd.exe
```

```
E:\Maven-project1\flipkart-app> java -cp target/flipkart-app-1.0-SNAPSHOT.jar com.mvit.App
```

```
C:\Windows\System32\cmd.exe
```

```
E:\Maven-project1\flipkart-app> java -cp target/flipkart-app-1.0-SNAPSHOT.jar com.mvit.App
```

```
20+30 = 50
```

```
Application executed sucessfully!
```

```
E:\Maven-project1\flipkart-app> _
```

The above command is used to **run a Java application** from the command line. Here's a breakdown of each part:

- **java**: This is the Java runtime command used to run Java applications.
- **-cp**: This stands for **classpath**, and it specifies the location of the classes and resources that the JVM needs to run the application. In this case, it's pointing to the JAR file where your compiled classes are stored.
- **target/flipkart-app-1.0-SNAPSHOT.jar**: This is the **JAR file** (Java ARchive) that contains the compiled Java classes and resources. It's located in the **target** directory, which Maven creates after you run `mvn package`.
- **com.mvit.App**: This is the **main class** that contains the `main()` method. When you run this command, Java looks for the `main()` method inside the `App` class located in the `com.example` package and executes it.

Creating a Basic Maven Web-Based Java Application Project

To create a Maven-based Java web application, the essential requirements include a JDK, Maven, a web server like Tomcat, an IDE, a web browser, and necessary dependencies. Setting up a proper environment ensures smooth project development and deployment.

1. java Development Kit (JDK)

- Java is the programming language used for web applications.
- JDK provides tools like the Java compiler and runtime environment.
- Install JDK 17 and set up the `JAVA_HOME` environment variable.

2. Apache Maven

- Maven helps automate project setup, dependency management, and builds.
- It ensures that the required libraries are downloaded and organized.
- Install Maven and verify its setup using `mvn -version`.

3. Web Server (Apache Tomcat)

- Tomcat is used to run Java web applications.
- It acts as a servlet container and processes web requests.
- Install Tomcat and start it to deploy and test the application.

4. Integrated Development Environment (IDE)

- An IDE makes coding easier with features like debugging and auto-completion.
- Examples: **Eclipse, IntelliJ IDEA, NetBeans**.
- Choose an IDE that supports Maven projects.

5. Web Browser

- A browser is required to access and test the web application.
- Examples: **Google Chrome, Mozilla Firefox, Microsoft Edge**.

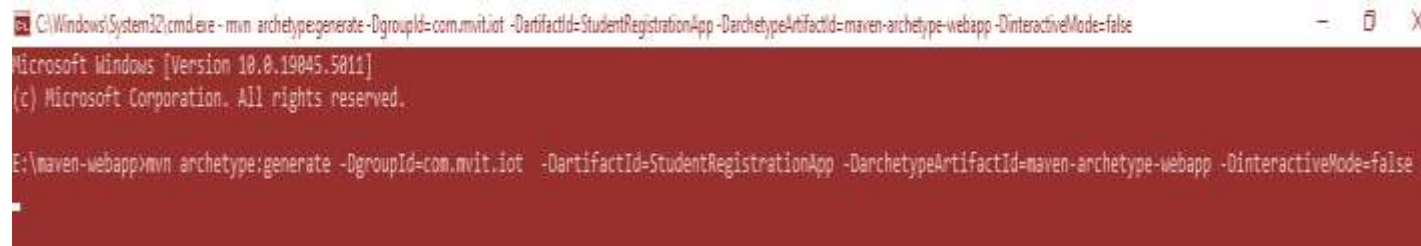
6. Maven Dependencies

- Dependencies are external libraries required for the web application.
- Important dependencies:
 - **Servlet API** (to handle web requests).
 - **JSP API** (for dynamic web pages).
 - **JSTL** (to simplify JSP coding).
- These dependencies are added in the `pom.xml` file.

Create a Maven Web Application

Step 1: Generate Maven Project

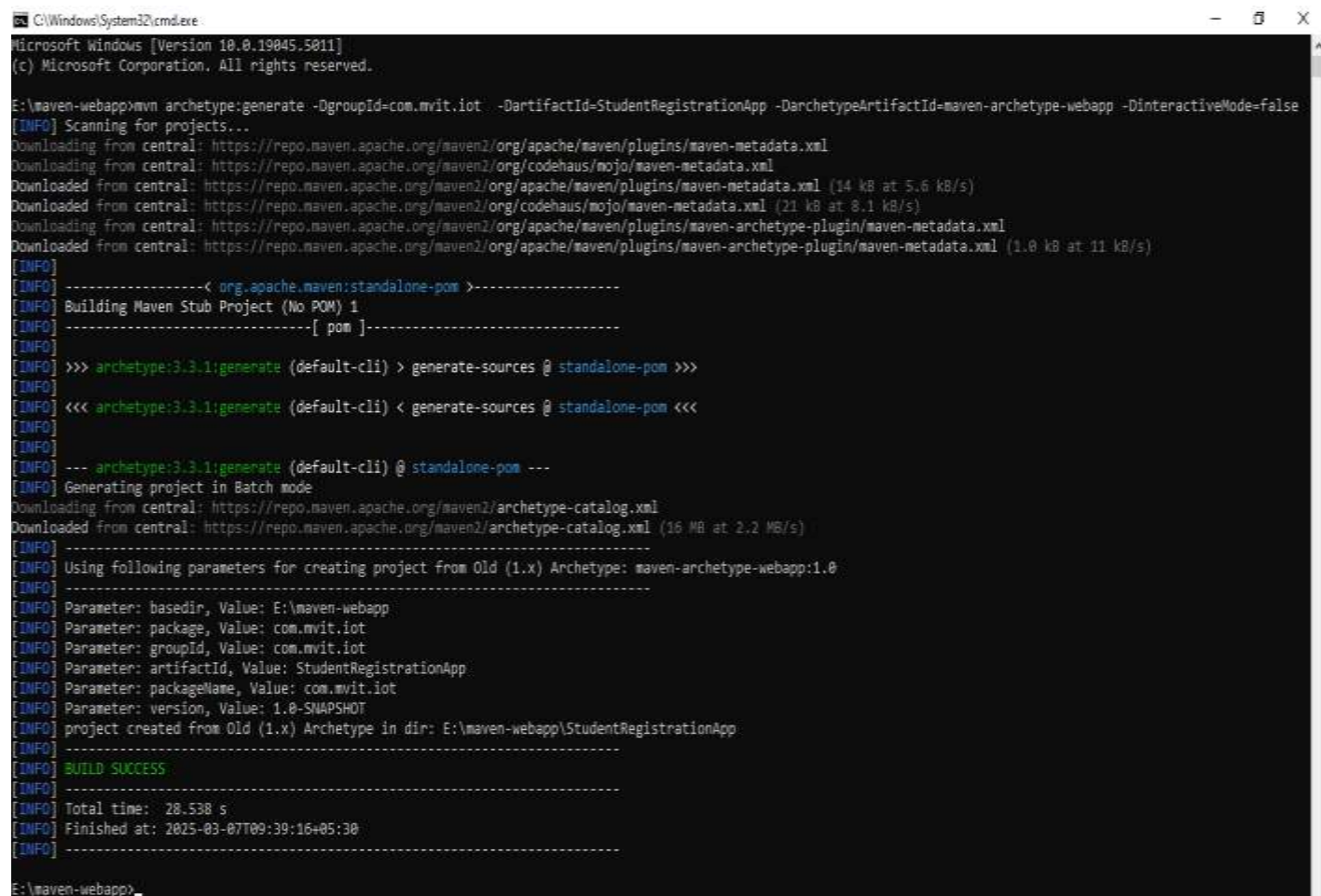
Use the Maven archetype to create a web project:



```
C:\Windows\System32\cmd.exe - mvn archetype:generate -DgroupId=com.mvit.iot -DartifactId=StudentRegistrationApp -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

E:\maven-webapp>mvn archetype:generate -DgroupId=com.mvit.iot -DartifactId=StudentRegistrationApp -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false
```

This will create a basic web application



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

E:\maven-webapp>mvn archetype:generate -DgroupId=com.mvit.iot -DartifactId=StudentRegistrationApp -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-metadata.xml
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/mojo/maven-metadata.xml
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-metadata.xml (14 kB at 5.6 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/mojo/maven-metadata.xml (21 kB at 8.1 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-archetype-plugin/maven-metadata.xml
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-archetype-plugin/maven-metadata.xml (1.0 kB at 11 kB/s)
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO] >>> archetype:3.3.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< archetype:3.3.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO] --- archetype:3.3.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (16 MB at 2.2 MB/s)
[INFO] -----
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO] -----
[INFO] Parameter: basedir, Value: E:\maven-webapp
[INFO] Parameter: package, Value: com.mvit.iot
[INFO] Parameter: groupId, Value: com.mvit.iot
[INFO] Parameter: artifactId, Value: StudentRegistrationApp
[INFO] Parameter: packageName, Value: com.mvit.iot
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: E:\maven-webapp\StudentRegistrationApp
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 20.538 s
[INFO] Finished at: 2025-03-07T09:39:16+05:30
[INFO] -----

E:\maven-webapp>
```

Project Structure

The generated project will have the following structure:

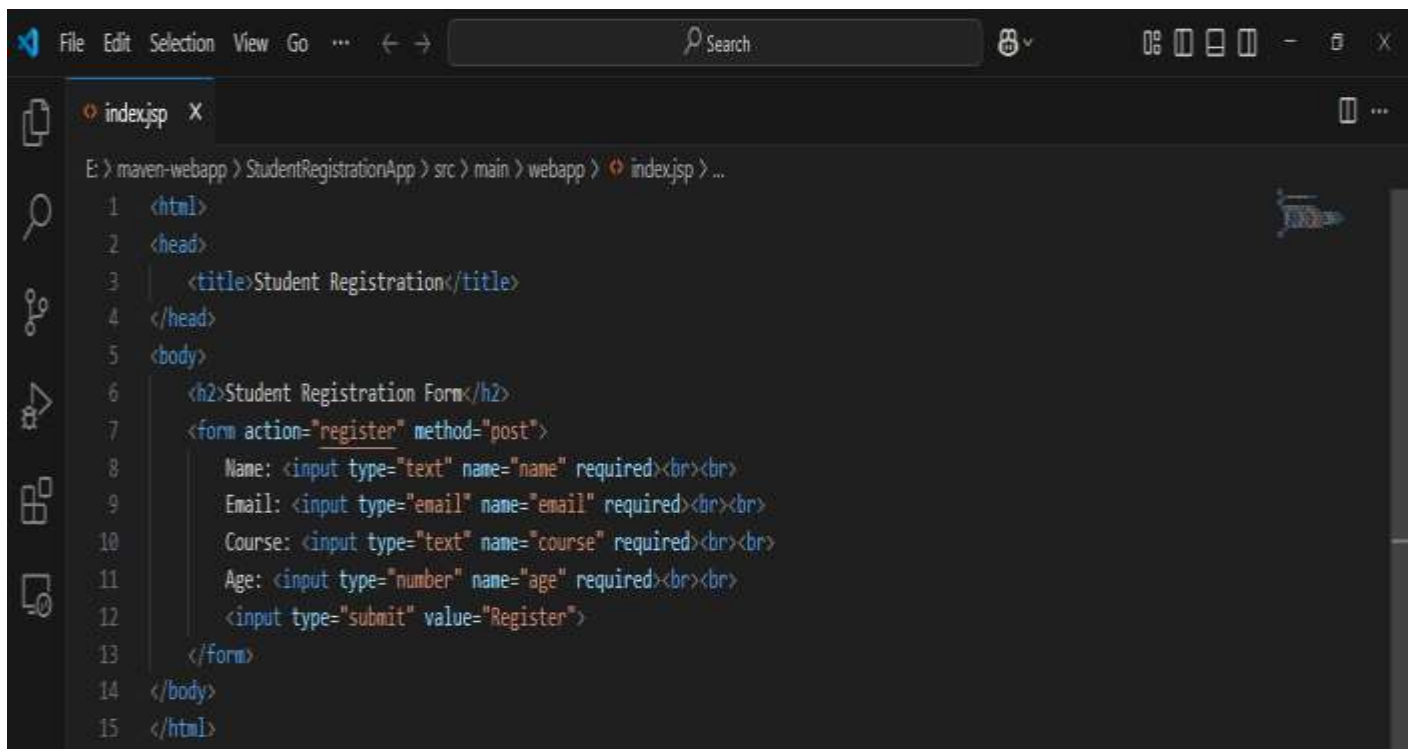
StudentRegistrationApp

```
| — src
|   | — main
|   |   | — java
|   |   | — resources
|   |   | — webapp
|   |   | — WEB-INF
|   |   |   | — web.xml
|   |   |   | — index.jsp
| — pom.xml
```

- **src/main/java** → Contains Java source files (Servlets, Controllers, etc.).
- **src/main/webapp** → Contains JSP files, HTML, CSS, and JS.
- **WEB-INF** → Holds configuration files like `web.xml`.
- **pom.xml** → Defines Maven dependencies and plugins.

Step 2 : Configure pom.xml**Add Tomcat Plugin for Running the Application**

```
<plugins>
  <plugin>
    <groupId>org.apache.tomcat.maven</groupId>
    <artifactId>tomcat7-maven-plugin</artifactId>
    <version>2.2</version>
    <configuration>
      <path></path>
      <port>8080</port>
    </configuration>
  </plugin>
</plugins>
</build>
</project>
```

Step 3: Create index.jsp (Student Registration Form)**Create index.jsp inside src/main/webapp/:**A screenshot of an IDE window showing the content of index.jsp. The file is located at E:\maven-webapp>StudentRegistrationApp>src>main>webapp>index.jsp. The code is an HTML form for student registration with fields for Name, Email, Course, and Age, and a Register button.

```
1 <html>
2 <head>
3   <title>Student Registration</title>
4 </head>
5 <body>
6   <h2>Student Registration Form</h2>
7   <form action="register" method="post">
8     Name: <input type="text" name="name" required><br><br>
9     Email: <input type="email" name="email" required><br><br>
10    Course: <input type="text" name="course" required><br><br>
11    Age: <input type="number" name="age" required><br><br>
12    <input type="submit" value="Register">
13  </form>
14 </body>
15 </html>
```

Step 4: Build and Run the Application

4.1 compile the project

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

E:\maven-webapp\StudentRegistrationApp> mvn compile
[INFO] Scanning for projects...
```

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

E:\maven-webapp\StudentRegistrationApp> mvn compile
[INFO] Scanning for projects...
[INFO] -----< com.mvit.iot:StudentRegistrationApp >-----
[INFO] Building StudentRegistrationApp Maven Webapp 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ war ]-----
[INFO] --- resources:3.3.1:resources (default-resources) @ StudentRegistrationApp ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO] --- compiler:3.13.0:compile (default-compile) @ StudentRegistrationApp ---
[INFO] No sources to compile
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.400 s
[INFO] Finished at: 2025-03-09T11:52:35+05:30
[INFO] -----

E:\maven-webapp\StudentRegistrationApp>
```

4.2 package the project

```
E:\maven-webapp\StudentRegistrationApp> mvn package
[INFO] Scanning for projects...

C:\Windows\System32\cmd.exe
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.23.0/commons-compress-1.23.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.23.0/commons-compress-1.23.0.pom (22 kB at 69 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/56/commons-parent-56.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/56/commons-parent-56.pom (82 kB at 208 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/junit-bom/5.9.1/junit-bom-5.9.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/junit-bom/5.9.1/junit-bom-5.9.1.pom (5.0 kB at 131 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/github/luben/zstd-jni/1.5.5-2/zstd-jni-1.5.5-2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/com/github/luben/zstd-jni/1.5.5-2/zstd-jni-1.5.5-2.pom (1.9 kB at 38 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-component-annotations/1.5.4/plexus-component-annotations-1.5.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-component-annotations/1.5.4/plexus-component-annotations-1.5.4.pom (815 B at 3.0 KB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-containers/1.5.4/plexus-containers-1.5.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-containers/1.5.4/plexus-containers-1.5.4.pom (4.2 kB at 27.7 KB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/2.0.5/plexus-2.0.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/2.0.5/plexus-2.0.5.pom (17 kB at 314 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.6.0/maven-archiver-3.6.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.6.0/maven-archiver-3.6.0.jar (26 kB at 282 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.4.0/plexus-io-3.4.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.4.0/plexus-io-3.4.0.jar (70 kB at 245 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-io/2.13.0/commons-io-2.13.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-io/2.13.0/commons-io-2.13.0.jar (221 kB at 90 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/4.7.1/plexus-archiver-4.7.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/4.7.1/plexus-archiver-4.7.1.jar (221 kB at 90 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.23.0/commons-compress-1.23.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.23.0/commons-compress-1.23.0.jar (1.1 MB at 100 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/github/luben/zstd-jni/1.5.5-2/zstd-jni-1.5.5-2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/com/github/luben/zstd-jni/1.5.5-2/zstd-jni-1.5.5-2.jar (5.9 MB at 250 kB/s)
[INFO] Packaging webapp
[INFO] Assembling webapp [StudentRegistrationApp] in [E:\maven-webapp\StudentRegistrationApp\target\StudentRegistrationApp]
[INFO] Processing war project
[INFO] Copying webapp resources [E:\maven-webapp\StudentRegistrationApp\src\main\webapp]
[INFO] Building war: E:\maven-webapp\StudentRegistrationApp\target\StudentRegistrationApp.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 41.869 s
[INFO] Finished at: 2025-03-09T11:57:31+05:30
[INFO] -----

E:\maven-webapp\StudentRegistrationApp>
```

This will generate a WAR file in the target/ directory.

4.3 Deploy and Run on Embedded Tomcat

Run the following command:

```
C:\Windows\System32\cmd.exe - mvn tomcat7:run

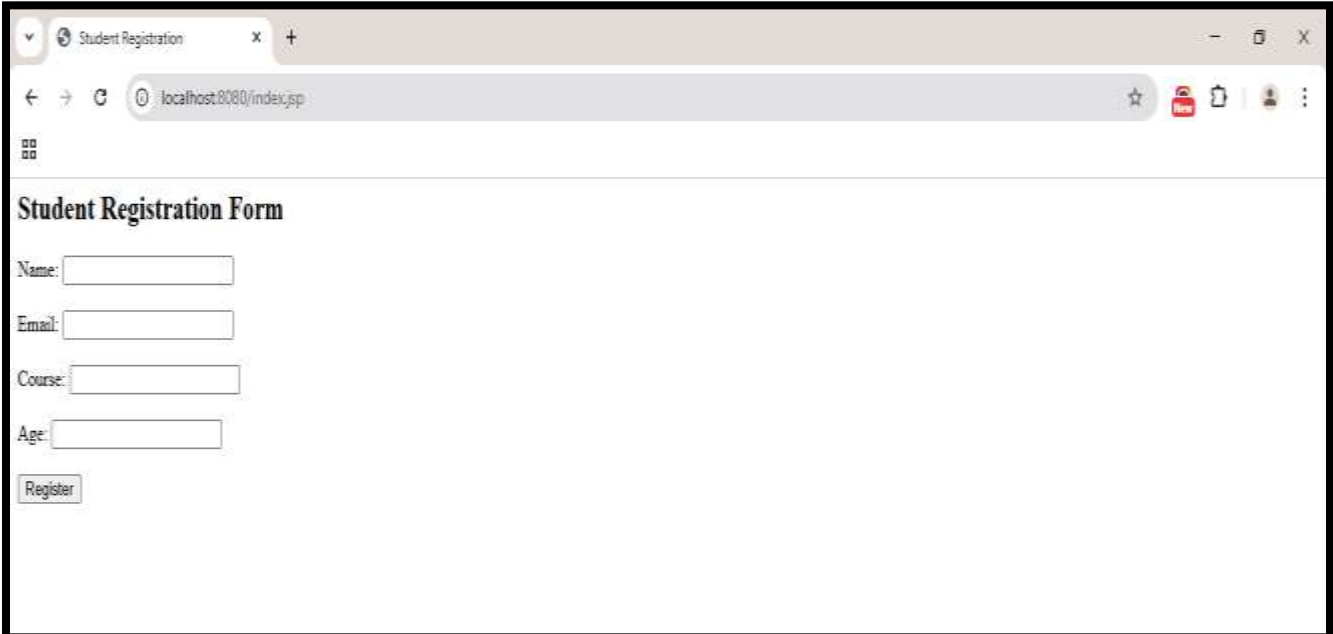
E:\maven-webapp\StudentRegistrationApp> mvn tomcat7:run
[INFO] Scanning for projects...
```

```
C:\Windows\System32\cmd.exe - mvn tomcat7:run

E:\maven-webapp\StudentRegistrationApp> mvn tomcat7:run
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.mvit.iot:StudentRegistrationApp >-----
[INFO] Building StudentRegistrationApp Maven Webapp 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] -----[ war ]-----
[INFO]
[INFO] >>> tomcat7:2.2:run (default-cli) > process-classes @ StudentRegistrationApp >>>
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ StudentRegistrationApp ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources; i.e. build is platform dependent!
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ StudentRegistrationApp ---
[INFO] No sources to compile
[INFO]
[INFO] <<< tomcat7:2.2:run (default-cli) < process-classes @ StudentRegistrationApp <<<
[INFO]
[INFO] --- tomcat7:2.2:run (default-cli) @ StudentRegistrationApp ---
[INFO] Running war on http://localhost:8080/
[INFO] Using existing Tomcat server configuration at E:\maven-webapp\StudentRegistrationApp\target\tomcat
[INFO] create webapp with contextPath:
Mar 09, 2025 12:20:28 PM org.apache.coyote.AbstractProtocol init
INFO: Initializing ProtocolHandler ["http-bio-8080"]
Mar 09, 2025 12:20:28 PM org.apache.catalina.core.StandardService startInternal
INFO: Starting service Tomcat
Mar 09, 2025 12:20:28 PM org.apache.catalina.core.StandardEngine startInternal
INFO: Starting Servlet Engine: Apache Tomcat/7.0.47
Mar 09, 2025 12:20:33 PM org.apache.catalina.util.SessionIdGenerator createSecureRandom
INFO: Creation of SecureRandom instance for session ID generation using [SHA1PRNG] took [162] milliseconds.
Mar 09, 2025 12:20:33 PM org.apache.coyote.AbstractProtocol start
INFO: Starting ProtocolHandler ["http-bio-8080"]
```

Step 5 : Open a browser and visit:

- <http://localhost:8080/index.jsp> → Student Registration Form



Student Registration

localhost:8080/index.jsp

Student Registration Form

Name:

Email:

Course:

Age:

3. Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation

Gradle Project Overview

1: Setting Up a Gradle Project

- To create a new Gradle project using the command line:

```
E:\Gradle-project1> gradle init
```

This command creates a new Java application project with a sample **build.gradle** file.

2. Understanding Build Scripts

Gradle uses a DSL (Domain-Specific Language) to define the build scripts. Gradle supports two DSLs:

- **Groovy DSL** (default)
- **Kotlin DSL** (alternative)

Groovy DSL: This is the default language used for Gradle build scripts (**build.gradle**).

Example of a simple **build.gradle** file (Groovy DSL):

```
plugins {  
    id 'java'  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-web:2.5.4'  
}
```

Kotlin DSL: Gradle also supports Kotlin for its build scripts (build.gradle.kts).

Example of a simple build.gradle.kts file (Kotlin DSL):

```
plugins {  
    id java  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation ("org.springframework.boot:spring-boot-starter-web:2.5.4")  
}
```

Difference between Groovy and Kotlin DSL:

- **Syntax:** Groovy uses a more concise, dynamic syntax, while Kotlin offers a more structured, statically-typed approach.
- **Error handling:** Kotlin provides better error detection at compile time due to its static nature.

3: Dependency Management

Dependencies in Gradle are defined inside the dependencies block

- Open build.gradle or build.gradle.kts.
- Add a dependency (e.g., **JUnit** for testing)

```
dependencies {  
    testImplementation 'junit:junit:4.13.2'  
}
```

4: Task Automation

Gradle tasks automate various tasks in your project lifecycle, like compiling code, running tests, and creating builds .

1. **Using predefined tasks:** Gradle provides many predefined tasks for common activities, such as:
 - **build** – compiles the project, runs tests, and creates the build output.
 - **test** – runs tests.
 - **clean** – deletes the build output.
2. Example of running the **build** task:

```
> gradle build
```

3. **Creating custom tasks:** You can define your own tasks to automate specific actions. For example, creating a custom task to print a message.

- **Example Groovy DSL:**

```
task printMessage {  
    doLast {  
        println 'This is a custom task automation'  
    }  
}
```

- **Example Kotlin DSL:**

```
tasks.register("printMessage") {  
    doLast {  
        println("This is a custom task automation")  
    }  
}
```

4. Running Gradle Tasks

- To run the **build** task: **gradle build**
- To run a custom task: **gradle printMessage**

Working with Gradle Project (Groovy DSL):

Step 1: Create a new Project

```
E:\Gradle-project1> gradle init
```

- while creating project it will ask necessary requirement:
 - **Enter target Java version (min: 7, default: 21):** 17
 - **Project name (default: program3-groovy):** groovyProject
 - **Select application structure:**
 - 1: Single application project
 - 2: Application and library project
 - **Enter selection (default: Single application project) [1..2]** 1
 - **Select build script DSL:**
 - 1: Kotlin
 - 2: Groovy
 - **Enter selection (default: Kotlin) [1..2]** 2
 - **Select test framework:**
 - 1: JUnit 4
 - 2: TestNG
 - 3: Spock
 - 4: JUnit Jupiter
 - **Enter selection (default: JUnit Jupiter) [1..4]** 1
 - **Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no]**
 - no

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

E:\Gradle-project1> gradle init

Select type of build to generate:
 1: Application
 2: Library
 3: Gradle plugin
 4: Basic (build structure only)
Enter selection (default: Application) [1..4] 1

Select implementation language:
 1: Java
 2: Kotlin
 3: Groovy
 4: Scala
 5: C++
 6: Swift
Enter selection (default: Java) [1..6] 1

Enter target Java version (min: 7, default: 21): 17

Project name (default: Gradle-project1): Groovy1

Select application structure:
 1: Single application project
 2: Application and library project
Enter selection (default: Single application project) [1..2] 1

Select build script DSL:
 1: Kotlin
 2: Groovy
Enter selection (default: Kotlin) [1..2] 2

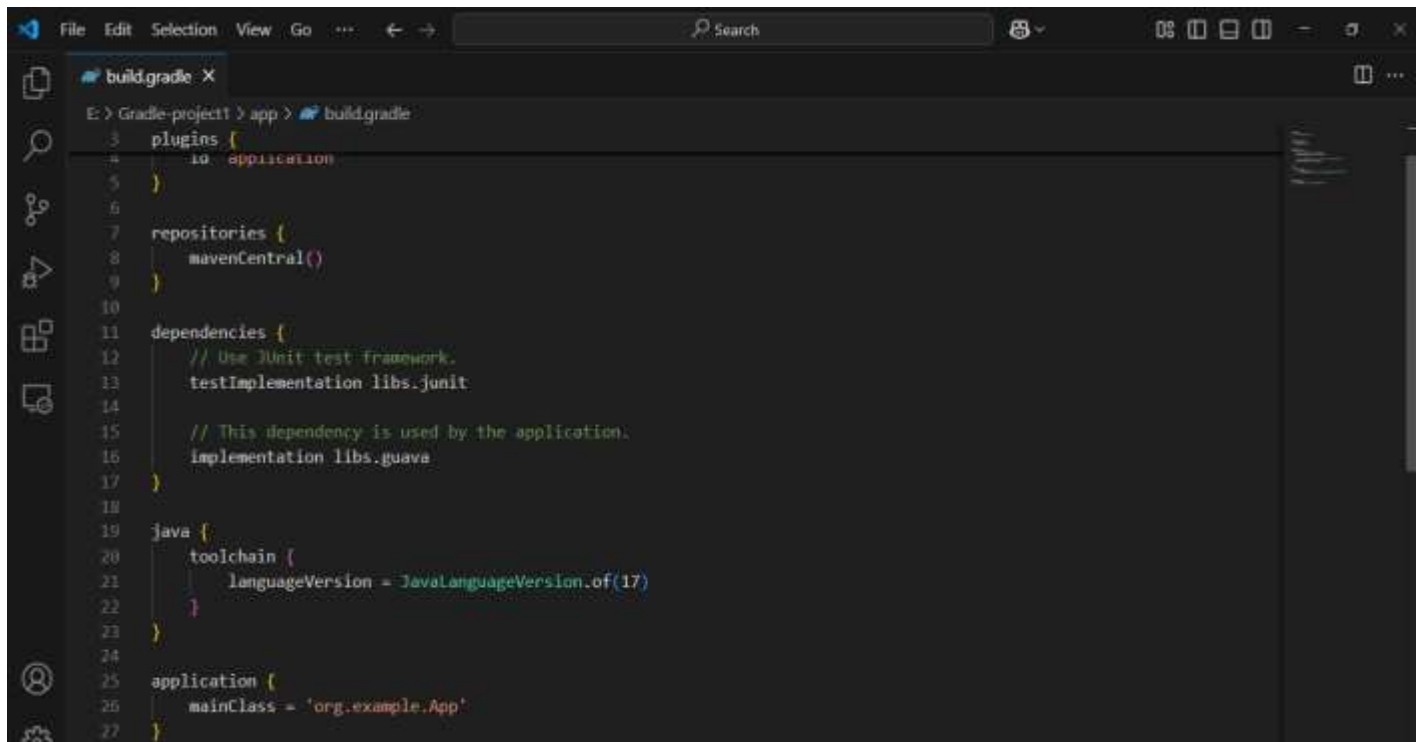
Select test framework:
 1: JUnit 4
 2: TestNG
 3: Spock
 4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 1

Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] no

> Task :init
Learn more about Gradle by exploring our Samples at https://docs.gradle.org/8.12.1/samples/sample_building_java_applications.html

BUILD SUCCESSFUL in 2h 3m 7s
1 actionable task: 1 executed
E:\Gradle-project1>
```

Step 2: build.gradle (Groovy DSL)

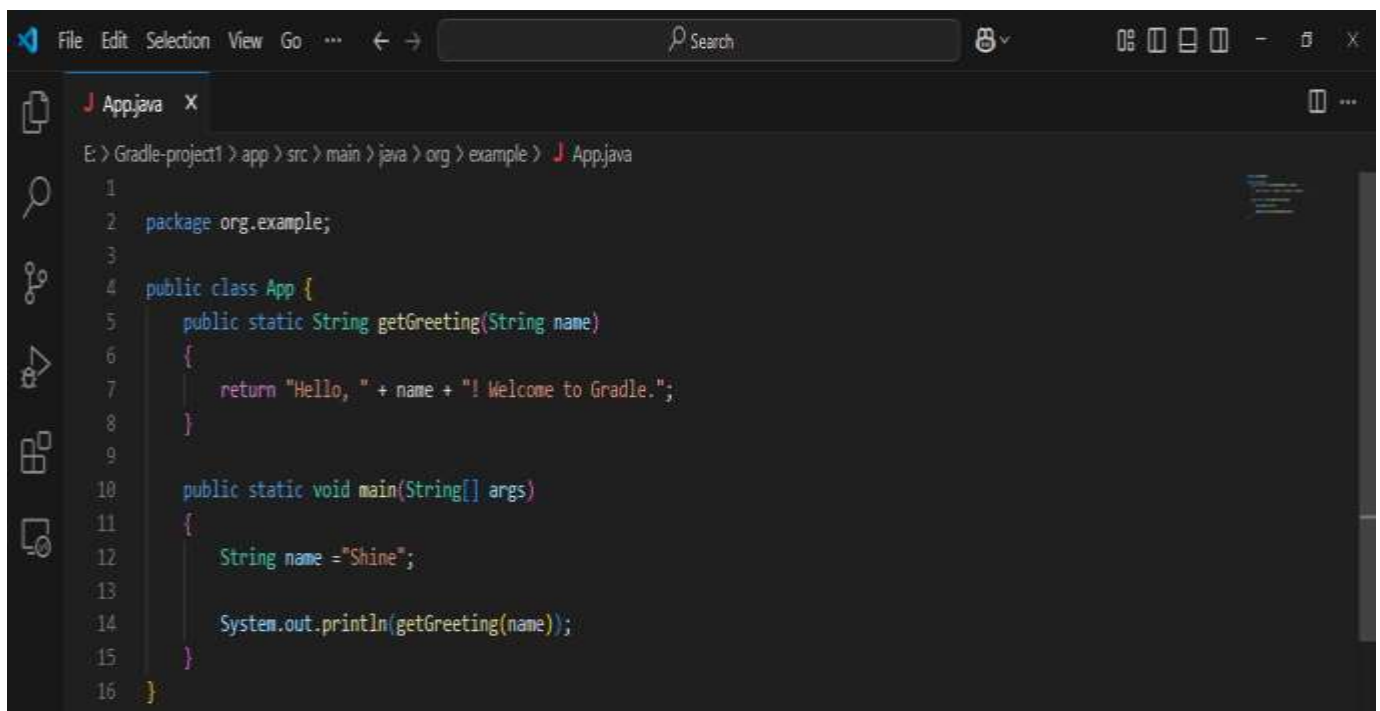


The screenshot shows an IDE window titled 'build.gradle' with the following content:

```
E:\> Gradle-project1 > app > build.gradle
3  plugins {
4      id 'application'
5  }
6
7  repositories {
8      mavenCentral()
9  }
10
11 dependencies {
12     // Use JUnit test framework.
13     testImplementation libs.junit
14
15     // This dependency is used by the application.
16     implementation libs.guava
17 }
18
19 java {
20     toolchain {
21         languageVersion = JavaLanguageVersion.of(17)
22     }
23 }
24
25 application {
26     mainClass = 'org.example.App'
27 }
```

Step 3: Manually navigate the folder path like src/main/java/org/example/

Write a simple program

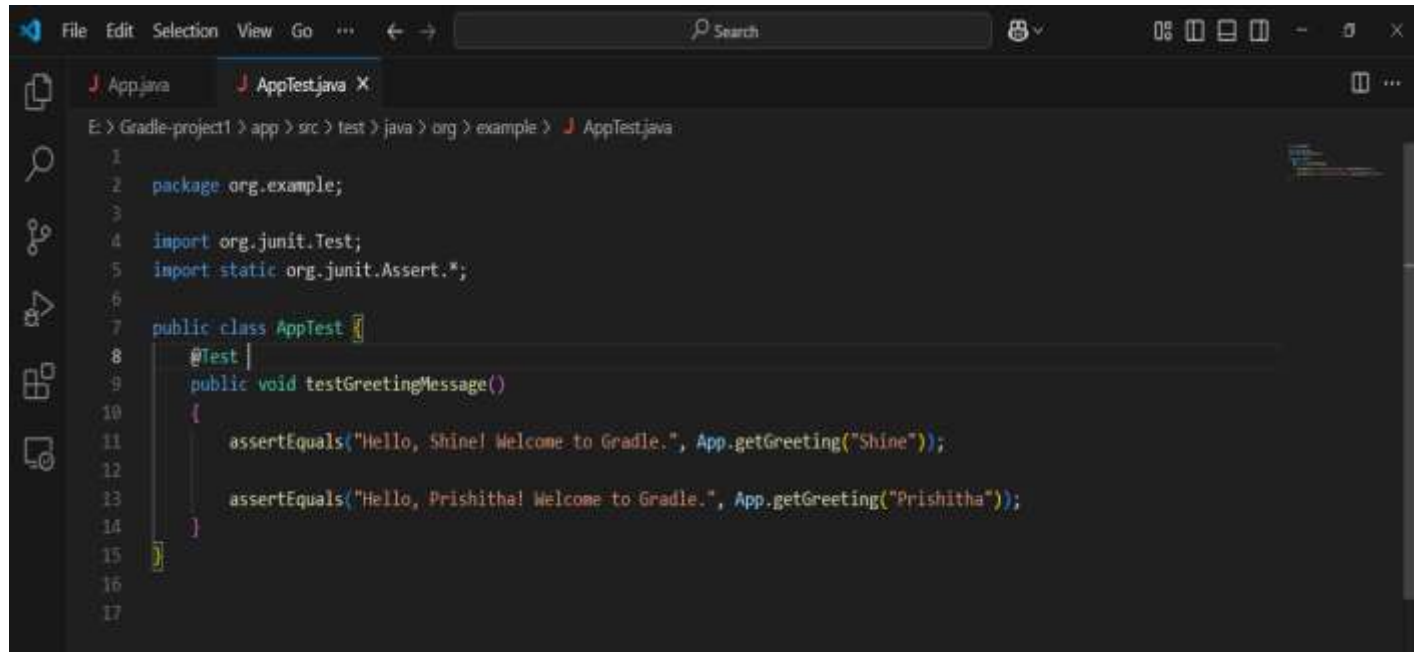


The screenshot shows an IDE window titled 'App.java' with the following content:

```
E:\> Gradle-project1 > app > src > main > java > org > example > App.java
1
2 package org.example;
3
4 public class App {
5     public static String getGreeting(String name)
6     {
7         return "Hello, " + name + "! Welcome to Gradle.";
8     }
9
10    public static void main(String[] args)
11    {
12        String name = "Shine";
13
14        System.out.println(getGreeting(name));
15    }
16 }
```

Step 4: AppTest.java (JUnitTest) (update the below code)

- Manually navigate the folder path like **src/test/java/org/example/**
- After then open that file and copy the below code and paste it, save it.



```
1 package org.example;
2
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6
7 public class AppTest {
8     @Test
9     public void testGreetingMessage()
10    {
11        assertEquals("Hello, Shine! Welcome to Gradle.", App.getGreeting("Shine"));
12
13        assertEquals("Hello, Prishitha! Welcome to Gradle.", App.getGreeting("Prishitha"));
14    }
15 }
16
17
```

Step 5 : Run Gradle Commands

- To build the project:

C:\Windows\System32\cmd.exe

```
E:\Gradle-project1>gradle build
```

C:\Windows\System32\cmd.exe

```
E:\Gradle-project1>gradle build
Reusing configuration cache.

BUILD SUCCESSFUL in 16s
7 actionable tasks: 7 executed
Configuration cache entry reused.
E:\Gradle-project1>
```

- To run the project:

 C:\Windows\System32\cmd.exe

```
E:\Gradle-project1> gradle run
```

 C:\Windows\System32\cmd.exe

```
E:\Gradle-project1> gradle run
```

```
Reusing configuration cache.
```

```
> Task :app:run
```

```
Hello, Shine! Welcome to Gradle.
```

```
BUILD SUCCESSFUL in 6s
```

```
2 actionable tasks: 2 executed
```

```
Configuration cache entry reused.
```

```
E:\Gradle-project1>
```

- To test the project

 C:\Windows\System32\cmd.exe

```
E:\Gradle-project1> gradle test
```

 C:\Windows\System32\cmd.exe

```
E:\Gradle-project1> gradle test
```

```
Calculating task graph as no cached configuration is available for tasks: test
```

```
BUILD SUCCESSFUL in 13s
```

```
3 actionable tasks: 2 executed, 1 up-to-date
```

```
Configuration cache entry stored.
```

```
E:\Gradle-project1> _
```

4. Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle

Step 1: Creating a Maven Project

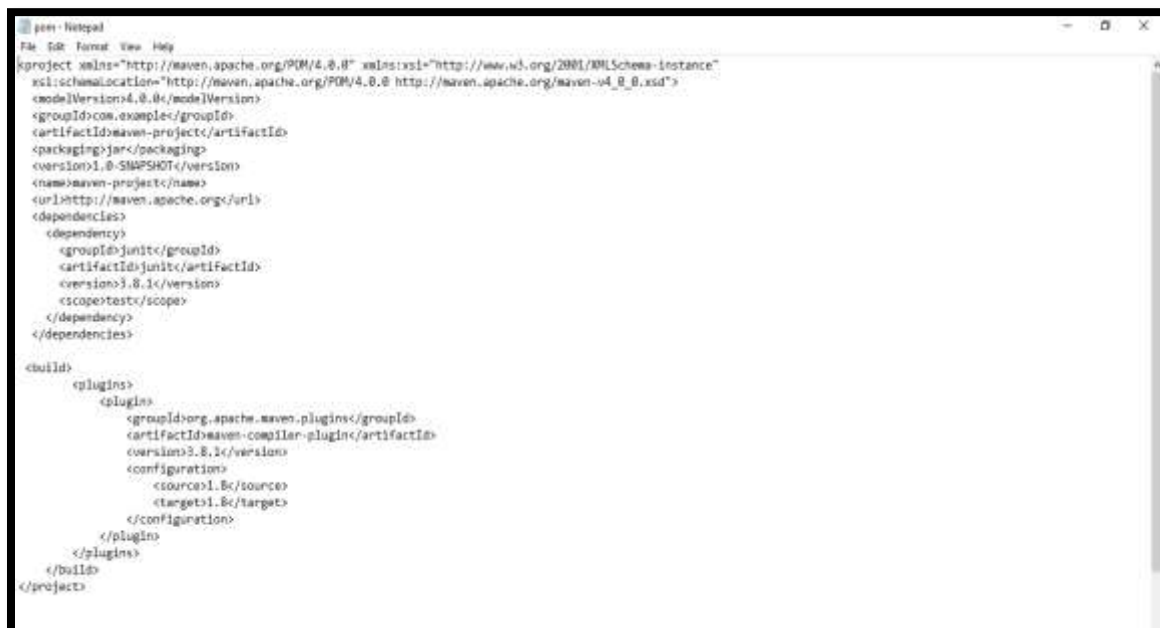
You can create a **Maven project** using the **mvn** command (or through your **IDE**, as mentioned earlier).

- **Using Command Line:**
 - To create a basic Maven project using the command line, you can use the following command

```
E:\Maven to Gradle>mvn archetype:generate -DgroupId=com.example -DartifactId=maven-project -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

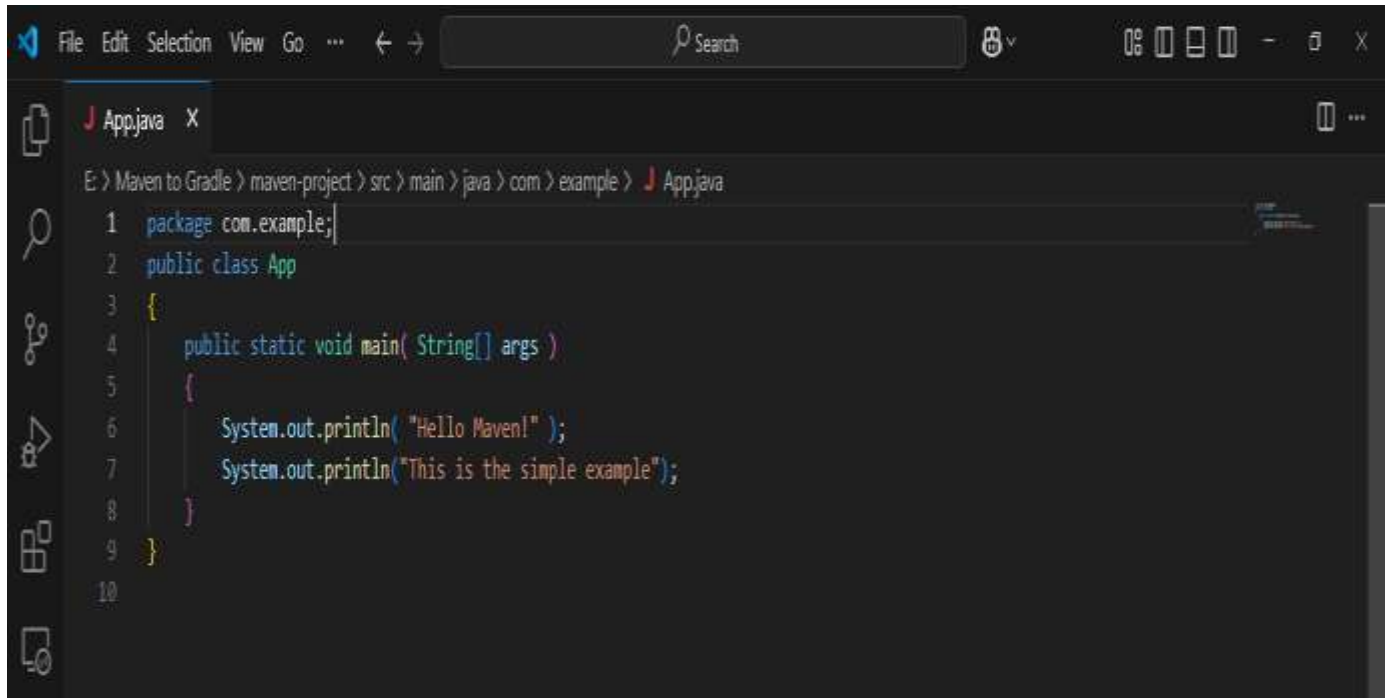
Step 2: Open The pom.xml File

- You can manually navigate the **project folder** named call **maven-example** and open the file pom.xml
- In case if you not getting project folder then type command in your cmd.
 - **cd maven-example** – is use to navigate the project folder.
 - **notepad pom.xml** – is use to open pom file in notepad.



Step 3: Open Java Code (App.java) File

- Open a file **App.java** inside the **src/main/java/com/example/** directory.
- After opening the **App.java** write a simple program



```
1 package com.example;
2 public class App
3 {
4     public static void main( String[] args )
5     {
6         System.out.println( "Hello Maven!" );
7         System.out.println("This is the simple example");
8     }
9 }
10
```

Note: before building the project make sure you are in the project folder if not navigate the project folder type command in your command prompt **cd maven-example**.

Step 4: Run the Project

To build and run this project, follow these steps:

- Open the terminal in the project directory and run the following command to build the project.

```
mvn compile
```



```

C:\Windows\System32\cmd.exe
E:\Maven to Gradle> cd maven-project
E:\Maven to Gradle\maven-project> mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:maven-project >-----
[INFO] Building maven-project 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] -----[ jar ]-----
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.8.1/maven-compiler-plugin-3.8.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.8.1/maven-compiler-plugin-3.8.1.pom (12 kB at 1.2 kB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/33/maven-plugins-33.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/33/maven-plugins-33.pom (11 kB at 56 kB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.8.1/maven-compiler-plugin-3.8.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.8.1/maven-compiler-plugin-3.8.1.jar (62 kB at 441 kB/s)
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ maven-project ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory E:\Maven to Gradle\maven-project\src\main\resources
[INFO]
[INFO] --- compiler:3.8.1:compile (default-compile) @ maven-project ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file to E:\Maven to Gradle\maven-project\target\classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 26.862 s
[INFO] Finished at: 2025-03-09T18:51:09+05:30
[INFO]
E:\Maven to Gradle\maven-project>

```

- package the project

```

C:\Windows\System32\cmd.exe
E:\Maven to Gradle\maven-project> mvn package
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:maven-project >-----
[INFO] Building maven-project 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ maven-project ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory E:\Maven to Gradle\maven-project\src\main\resources
[INFO]
[INFO] --- compiler:3.8.1:compile (default-compile) @ maven-project ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ maven-project ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory E:\Maven to Gradle\maven-project\src\test\resources
[INFO]
[INFO] --- compiler:3.8.1:testCompile (default-testCompile) @ maven-project ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file to E:\Maven to Gradle\maven-project\target\test-classes
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ maven-project ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit3.JUnit3Provider
[INFO]
[INFO] T E S T S
[INFO]
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.120 s -- in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

```

- Run the program with below command:

```
C:\Windows\System32\cmd.exe
[INFO] --- surefire:3.2.5:test (default-test) @ maven-project ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit3.JUnit3Provider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.120 s -- in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ maven-project ---
[INFO] Building jar: E:\Maven to Gradle\maven-project\target\maven-project-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 18.302 s
[INFO] Finished at: 2025-03-09T19:02:13+05:30
[INFO] -----
E:\Maven to Gradle\maven-project> java -cp target/maven-project-1.0-SNAPSHOT.jar com.example.App
Hello Maven!
This is the simple example
E:\Maven to Gradle\maven-project> █
```

Step 5: Migrate the Maven Project to Gradle

1. Initialize Gradle

```
E:\Maven to Gradle\maven-project> gradle init
```

- It will ask **Found a Maven build. Generate a Gradle build from this? (default: yes) [yes, no]**
 - Type Yes
- **Select build script DSL:**
 - 1: Kotlin
 - 2: Groovy
 - Enter selection (default: Kotlin) [1..2]
 - Type 2
- **Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no]**
 - Type No

 C:\Windows\System32\cmd.exe

```
E:\Maven to Gradle\maven-project> gradle init
```

```
Found a Maven build. Generate a Gradle build from this? (default: yes) [yes, no]    yes
```

```
Select build script DSL:
```

```
1: Kotlin
```

```
2: Groovy
```

```
Enter selection (default: Kotlin) [1..2]  2
```

```
Generate build using new APIs and behavior (some features may change in the next minor rel
```

```
> Task :init
```

```
Maven to Gradle conversion is an incubating feature.
```

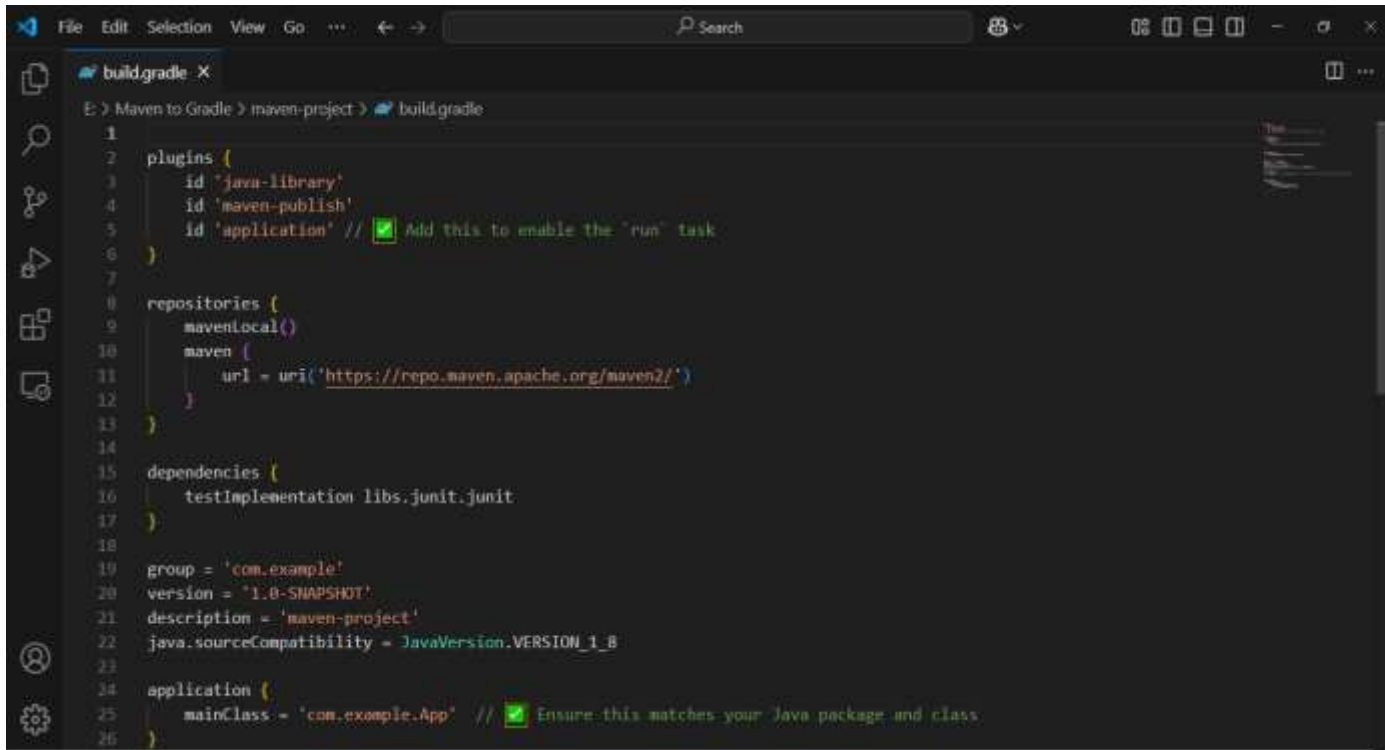
```
For more information, please refer to https://docs.gradle.org/8.12.1/userguide/migrating\_f
```

```
BUILD SUCCESSFUL in 4m 55s
```

```
1 actionable task: 1 executed
```

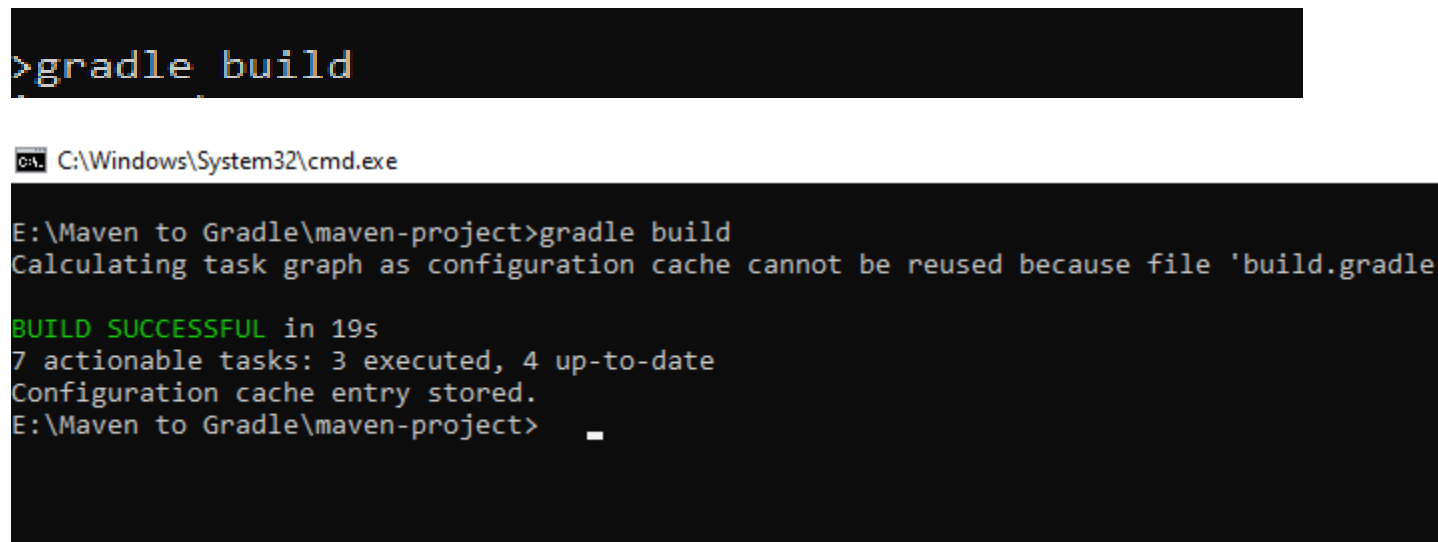
```
E:\Maven to Gradle\maven-project> █
```

2. Navigate the project folder and open `build.gradle` file then the do the necessary changes and save it.



Step 6: Run the Gradle Project

- **Build the Project:** run the below command to build the project.



- **Run the Application:** Once the build is successful, run the application using below command

C:\Windows\System32\cmd.exe

```
E:\Maven to Gradle\maven-project> gradle run
Calculating task graph as configuration cache cannot be reused because file 'build.gradle'
> Task :run
Hello Maven!
This is the simple example

BUILD SUCCESSFUL in 5s
2 actionable tasks: 1 executed, 1 up-to-date
Configuration cache entry stored.
E:\Maven to Gradle\maven-project> _
```

Step 7: Verify the Migration

Compare the Output: Make sure that both the **Maven** and **Gradle** builds produce the same output

- **Maven Output:**

```
E:\Maven to Gradle\maven-project> java -cp target/maven-project-1.0-SNAPSHOT.jar com.examp
Hello Maven!
This is the simple example

E:\Maven to Gradle\maven-project> _
```

- **Gradle Output:**

C:\Windows\System32\cmd.exe

```
E:\Maven to Gradle\maven-project> gradle run
Calculating task graph as configuration cache cannot be reused because file 'build.gradle' has changed.
> Task :run
Hello Maven!
This is the simple example

BUILD SUCCESSFUL in 5s
2 actionable tasks: 1 executed, 1 up-to-date
Configuration cache entry stored.
E:\Maven to Gradle\maven-project> _
```


Program 5

Introduction to Jenkins: What is Jenkins?, Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use

Introduction to Jenkins

What is Jenkins?

Jenkins is an open-source automation server widely used in the field of Continuous Integration (CI) and Continuous Delivery (CD). It allows developers to automate the building, testing, and deployment of software projects, making the development process more efficient and reliable

Key features of Jenkins:

- **CI/CD:** Jenkins supports Continuous Integration and Continuous Deployment, allowing developers to integrate code changes frequently and automate the deployment of applications.
- **Plugins:** Jenkins has a vast library of plugins that can extend its capabilities. These plugins integrate Jenkins with version control systems (like Git), build tools (like Maven or Gradle), testing frameworks, deployment tools, and much more.
- **Pipeline as Code:** Jenkins allows the creation of pipelines using Groovy-based DSL scripts or YAML files, enabling version-controlled and repeatable pipelines.
- **Cross-platform:** Jenkins can run on various platforms such as Windows, Linux, macOS, and others.

Installing Jenkins

Jenkins can be installed on local machines, on a cloud environment, or even in containers. Here's how you can install Jenkins in Window local System environments:

1. Installing Jenkins Locally (Windows)

A. Prerequisites:

- Ensure that **Java (JDK)** is installed on your system
- You can check if Java is installed by running **java -version** in the terminal.

B . Install Jenkins on Window System:

- Download the Jenkins Windows installer from the [official Jenkins website](#).
- Run the installer and select **Run as Local System** (not recommended for production).
- Choose a port (default: **8080**, or custom like **3030**) then **click on test** and **next**.
- Set Java path (e.g., C:\Program Files\Java\jdk-17\).
- Click **Next** until installation starts.
- After successfully installed, Jenkins will be accessed at `http://localhost:8080` or `http://localhost:3030`.

2. Jenkins Setup in browser:

After opening the browser and visiting your local Jenkins address (`http://localhost:8080` or your configured port), the **Jenkins setup page** will appear.

Jenkins Initial Setup Screen

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

`C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

Continue

3. Unlocking Jenkins (Administrator Password Required)

1. Upon accessing Jenkins in the browser, an **Administrator Password** is required. Navigate to the specified path: C:\Program Files\Jenkins\secrets\initialAdminPassword.

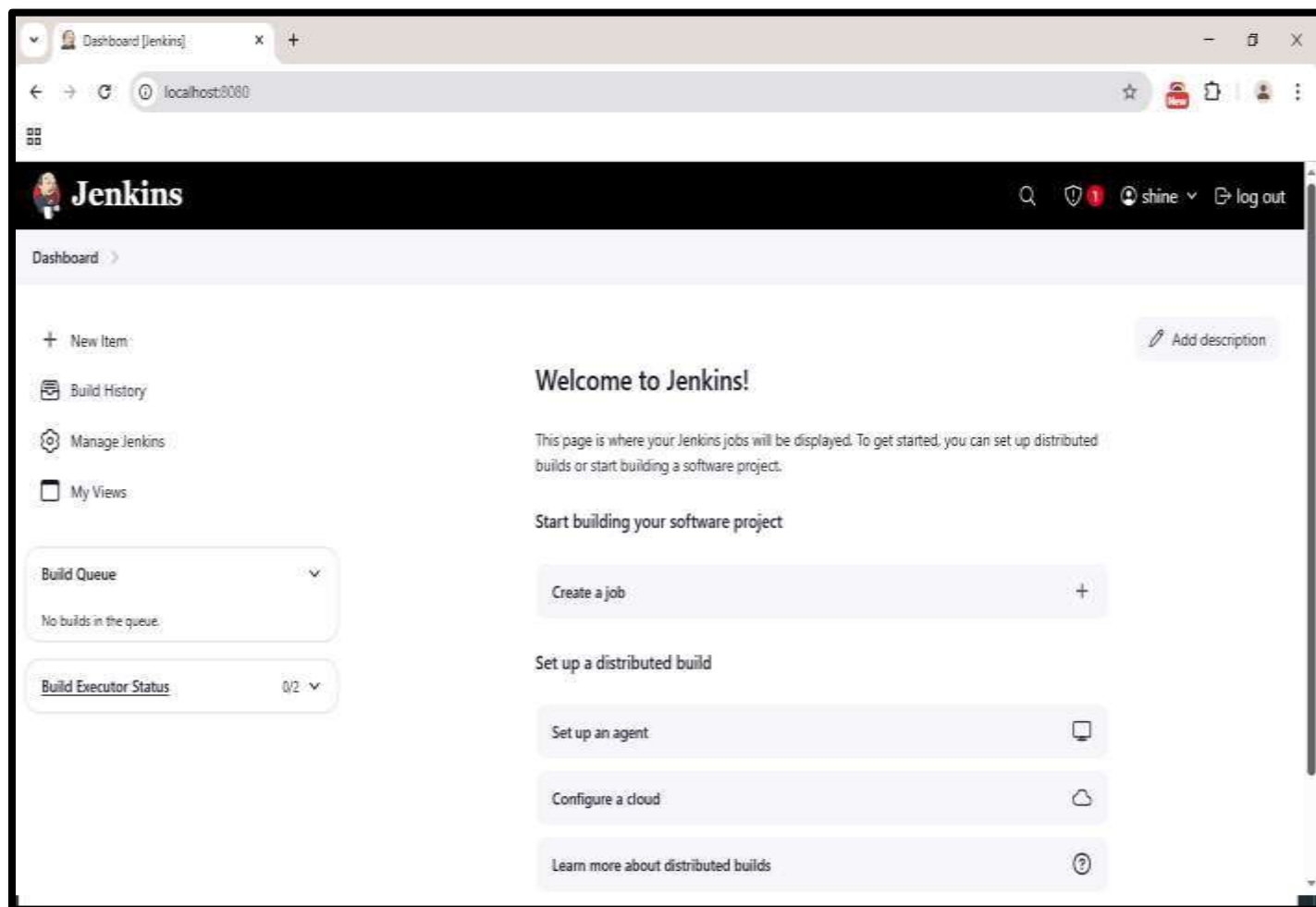
Open the file using **Notepad** or any text editor, copy the password, and paste it into the Jenkins setup page. Click **Continue** to proceed.

2. Next, Jenkins prompts for plugin installation. Select **Install Suggested Plugins** to automatically install the necessary plugins.

3. Once the plugins are installed, create an **Administrator Profile** by entering the required details. Click **Save and Continue**, then **Save and Finish**.

4. Finally, click **Start Using Jenkins** to complete the setup.

Jenkins Dashboard After Successful Installation and Set



Program - 6

Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests

Continuous Integration with Jenkins Objectives:

1. Set up a CI pipeline using Jenkins
2. Integrate Jenkins with Maven or Gradle
3. Run automated builds and unit tests

Tools/Software used:

- Jenkins (installed & running)
- Maven or Gradle
- Git
- JDK 17
- A sample Maven project

Steps to Execute the Lab Program :

Step 1: Install Jenkins and Required Tools

- Install Jenkins from <https://jenkins.io>
- Install JDK and Maven/Gradle
- Configure environment variables (JAVA_HOME, MAVEN_HOME, etc.)
- Start Jenkins (<http://localhost:8080>)

Step 2: Install Jenkins Plugins

- Go to **Manage Jenkins → Plugins**
- In the available tab search and Install:
 - Git Plugin
 - Maven Integration Plugin
 - Gradle Plugin (if using Gradle)
 - Pipeline plugin

Step 3: Create a Sample Java Project

- Create a Maven/Gradle Java project
- Login into Github account , create a new Github repository
- Push the project into Github repository.

Step 4: Create a Jenkins Job (Freestyle Project)

- Go to Jenkins Dashboard → **New Item**
- Name it (e.g., `Maven-CI-Pipeline`)
- Select **Freestyle project** → OK

Step 5: Configure Source Code Management

- Under **Source Code Management**, select **Git**
- Enter GitHub repo URL
Example: <https://github.com/username/sample-maven-project.git>
- In Branches to build section change master to main branch

Step 6: Set Build Triggers

- Click **Poll SCM**
- Schedule: `H/5 * * * *` (polls every 5 mins)
(Optional: You can also use "GitHub hook trigger" if webhook is configured)

Step 7: Configure Build

- Under **Build**, click **Add build step** → **Invoke top-level Maven targets**
 - Goals: `clean compile test package`
- **Add Another build step** → **Execute Windows batch command**
`java -cp target/<your-jar-name>.jar <your-main-class>`

Example : `java -cp target/sample-ci-project-1.0-SNAPSHOT.jar com.example.App`

- Click save

Step 8: Run and Verify

- Click **Build Now**
- Check **Console Output** for logs

Confirm:

- Source code is pulled from GitHub
- Project builds successfully
- Unit tests run
- JAR file is archived

Output Screenshots:

Pushing Maven Project to GitHub Using Git Bash

```
MINGW64:/e/maven-lab1/amazon-app
D:\E\DELL-PC\MINGW64 /e/maven-lab1/amazon-app
$ git init
Initialized empty Git repository in E:/maven-lab1/amazon-app/.git/

D:\E\DELL-PC\MINGW64 /e/maven-lab1/amazon-app (master)
$ git add .
warning: in the working copy of 'target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst', LF will be replaced by CRLF the next time Git touches it

D:\E\DELL-PC\MINGW64 /e/maven-lab1/amazon-app (master)
$ git commit -m "first commit"
[master (root commit): 16c0560] first commit
13 files changed, 191 insertions(+)
create mode 100644 pom.xml
create mode 100644 src/main/java/com/mvit/App.java
create mode 100644 src/test/java/com/mvit/AppTest.java
create mode 100644 target/amazon-app-1.0-SNAPSHOT.jar
create mode 100644 target/classes/com/mvit/App.class
create mode 100644 target/maven-archiver/pom.properties
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
create mode 100644 target/surefire-reports/TEST-com.mvit.AppTest.xml
create mode 100644 target/surefire-reports/com.mvit.AppTest.txt
create mode 100644 target/test-classes/com/mvit/AppTest.class

D:\E\DELL-PC\MINGW64 /e/maven-lab1/amazon-app (master)
$ git branch -M main

D:\E\DELL-PC\MINGW64 /e/maven-lab1/amazon-app (main)
$ git remote add origin https://github.com/gayithrishn/Mavenproject-Jenkins-CI-Pipeline.git

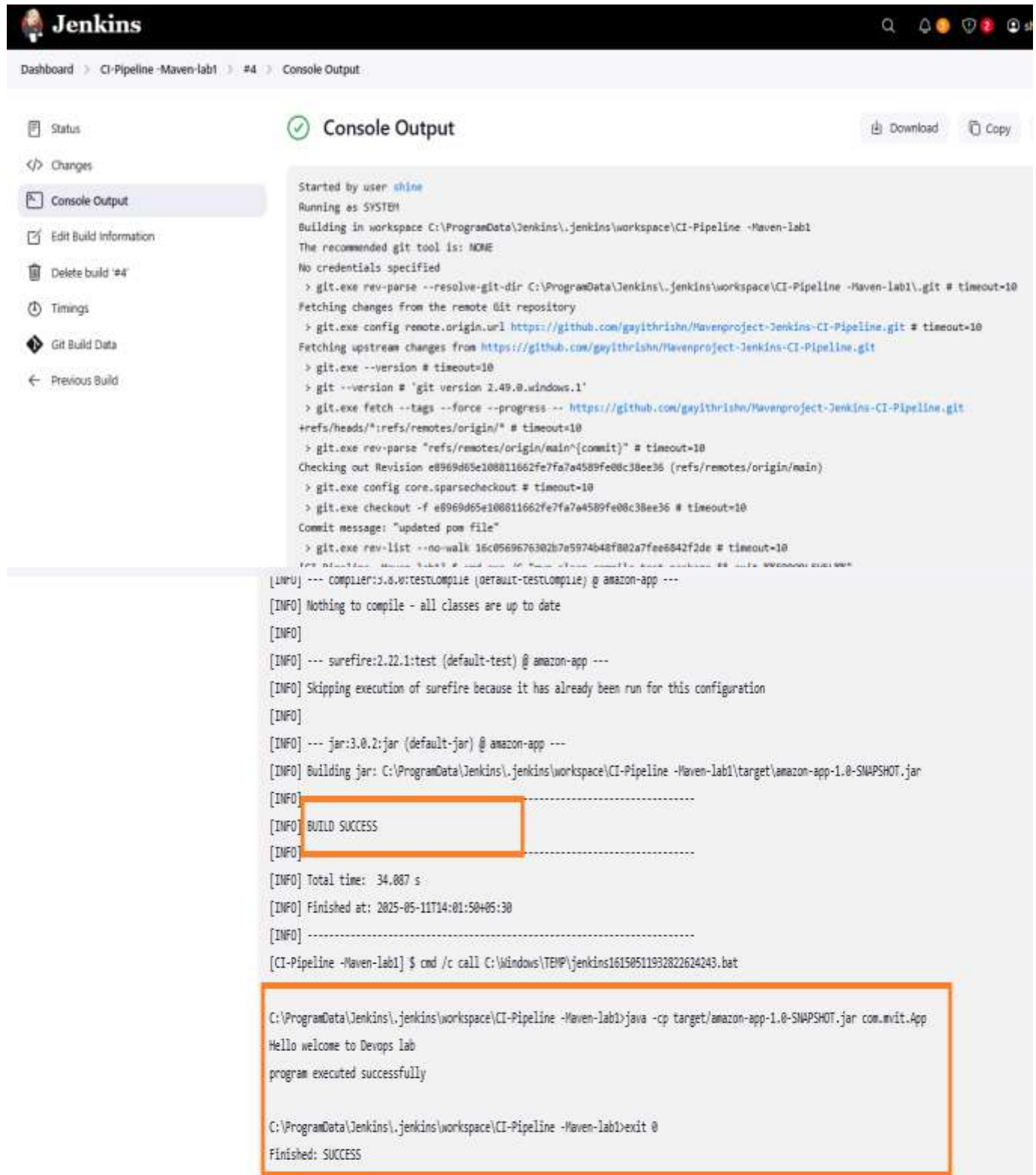
D:\E\DELL-PC\MINGW64 /e/maven-lab1/amazon-app (main)
$ git push -u origin main
Enumerating objects: 59, done.
```

GitHub Repository After Pushing Maven Project

The screenshot shows the GitHub interface for a repository named 'Mavenproject-Jenkins-CI-Pipeline' owned by 'gayithrishn'. The repository is public. The main branch is 'main', with 1 branch and 0 tags. A search bar is present with the text 'Go to file'. There are buttons for 'Add file' and 'Code'. Below the repository header, a commit by 'gayithrishn' is shown, labeled 'first commit', with a commit hash '16c0569' and a timestamp '3 minutes ago'. The commit details show three files: 'src', 'target', and 'pom.xml', all added in the 'first commit' 3 minutes ago.

File	Commit	Time
src	first commit	3 minutes ago
target	first commit	3 minutes ago
pom.xml	first commit	3 minutes ago

Jenkins Console Output After Build Execution



The screenshot shows the Jenkins web interface. The breadcrumb navigation is: Dashboard > CI-Pipeline -Maven-lab1 > #4 > Console Output. The left sidebar contains links for Status, Changes, Console Output (selected), Edit Build Information, Delete build '#4', Timings, Git Build Data, and Previous Build. The main area is titled 'Console Output' with a green checkmark icon and buttons for Download and Copy. The console output text is as follows:

```

Started by user shine
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\CI-Pipeline -Maven-lab1
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\CI-Pipeline -Maven-lab1\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/gayithrishn/Mavenproject-Jenkins-CI-Pipeline.git # timeout=10
Fetching upstream changes from https://github.com/gayithrishn/Mavenproject-Jenkins-CI-Pipeline.git
> git.exe --version # timeout=10
> git --version # 'git version 2.49.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/gayithrishn/Mavenproject-Jenkins-CI-Pipeline.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision e8969d65e108811662fe7fa7a4589fe08c38ee36 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f e8969d65e108811662fe7fa7a4589fe08c38ee36 # timeout=10
Commit message: "updated pom file"
> git.exe rev-list --no-walk 16c0569676302b7e5974b48f802a7fee6842f2de # timeout=10
[INFO] --- compiler:3.8.0:testCompile (default-testCompile) @ amazon-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:2.22.1:test (default-test) @ amazon-app ---
[INFO] Skipping execution of surefire because it has already been run for this configuration
[INFO]
[INFO] --- jar:3.0.2:jar (default-jar) @ amazon-app ---
[INFO] Building jar: C:\ProgramData\Jenkins\jenkins\workspace\CI-Pipeline -Maven-lab1\target\amazon-app-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 34.087 s
[INFO] Finished at: 2025-05-11T14:01:50+05:30
[INFO]
[CI-Pipeline -Maven-lab1] $ cmd /c call C:\Windows\TEMP\jenkins16150511932822624243.bat

C:\ProgramData\Jenkins\jenkins\workspace\CI-Pipeline -Maven-lab1>java -cp target\amazon-app-1.0-SNAPSHOT.jar com.mvit.App
Hello welcome to Devops lab
program executed successfully

C:\ProgramData\Jenkins\jenkins\workspace\CI-Pipeline -Maven-lab1>exit 0
Finished: SUCCESS

```

Program - 7

Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook.

Configuration Management :

Configuration Management means setting up and maintaining systems in a consistent and automated way. It ensures systems work reliably across multiple servers. In DevOps, tools like Ansible help automate these tasks to manage large numbers of systems easily.

Ansible:

Ansible is an open-source automation tool used for **configuration management**, **application deployment**, and **task automation**. It allows you to manage systems and software configurations across multiple servers simultaneously.

Ansible uses simple **human-readable YAML (Yet Another Markup Language)** files to describe automation tasks, known as **Playbooks**.

Components of Ansible :

- **Inventory:** Ansible uses an inventory file to define the list of hosts (servers) that it will manage.
- **Playbooks:** A Playbook is a file where tasks are defined using YAML syntax. A playbook specifies the actions that should be carried out on a target server or set of servers.
- **Modules:** Ansible uses modules to perform specific tasks (such as installing packages, starting services, etc.). Some of the most commonly used modules include the **apt** module for package management and the **service** module for service management.

Lab Setup:**Step 1: Install WSL on Windows :**

- Install **Windows Subsystem for Linux (WSL)** to provide a Linux environment on a Windows system.

- ❖ Open **PowerShell** as Administrator.
- ❖ Run the command:

wsl - - install

Step 2: Install Ubuntu on WSL

1. Open **Microsoft Store**
2. Search for "**Ubuntu App**" (choose **Ubuntu 22.04 LTS**)
3. Click **Install**
4. After installation, open Ubuntu from Start Menu
5. It will ask to create a UNIX username and password

Step 3: Update Ubuntu Packages

- Run the following command: **sudo apt update -y**

Step 4: Install Ansible inside WSL (Ubuntu)

- Run the command : **sudo apt install ansible -y**
- Verify the installation by checking the Ansible version : **ansible - -version**

Lab Execution : Configuration Management with Ansible**Step 5: Create a Working Directory**

- Run the following commands to create and navigate into the directory:

mkdir ansible-lab
cd ansible-lab

Step 6: Create the Inventory File

- Create a file that lists the target hosts for Ansible (in this case, the localhost).
- Use `vi` or any text editor to create the **inventory.ini** file:
vi inventory.ini
- Add the following content to the file:

```
[local]
localhost ansible_connection=local
```

- Save and exit by pressing **Esc**, typing **:wq**, and pressing **Enter**.

Step 7: Test Connectivity with Ansible

- Ensure that Ansible can communicate with the localhost.
- Run the following Ad-hoc command to test connectivity

```
ansible all -i inventory.ini -m ping
```

- Expected Output:

```
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

This confirms that Ansible can communicate with the localhost.

Step 8: Create a Basic Playbook

- Write a playbook to automate the installation of the **htop** package.
- Create a playbook `install_htop.yml`:

```
vi install_htop.yml
```

- Add the following content:

```
---
- name: Install htop system monitor tool
  hosts: local
  become: true

  tasks:
    - name: Install htop package
      ansible.builtin.package:
        name: htop
        state: present
```

Save and exit by pressing **Esc**, typing **:wq**, and pressing **Enter**.

Step 9: Run the Playbook

- Run the playbook to execute the automated tasks.
- Run the playbook with the following command:

```
ansible-playbook -i inventory.ini install_htop.yml
```

- Expected Output:

```
TASK [Ensure htop is installed] *****
changed: [localhost]
```

Step 10: Verify the Installation

- Verify that the **htop** package is installed correctly.
- Type the command
htop
- This should open the interactive **htop** system monitor (press **q** to exit it).

Output Screenshots :

Ansible Inventory File with Localhost Entry

```
prishitha@DELL-PC:~/devops-lab$ vi inventory.ini
[local]
localhost ansible_connection=local
```

Ansible Connectivity Check Using Ping Module

```
prishitha@DELL-PC:~/devops-lab$ vi inventory.ini
prishitha@DELL-PC:~/devops-lab$ ansible all -i inventory.ini -m ping
localhost | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
prishitha@DELL-PC:~/devops-lab$
```

Playbook File Created to Install htop (install_htop.yml)

```
prishitha@DELL-PC:~/devops-lab$ vi install_htop.yml
---
- name: Install htop system monitor tool
  hosts: local
  become: true
  tasks:
    - name: install htop package
      ansible.builtin.package:
        name: htop
        state: present
```

Playbook Execution Output for Installing htop

```
prishitha@DELL-PC:~/devops-lab$ vi install_htop.yml
prishitha@DELL-PC:~/devops-lab$ ansible-playbook -i inventory.ini install_htop.yml

PLAY [Install htop system monitor tool] *****

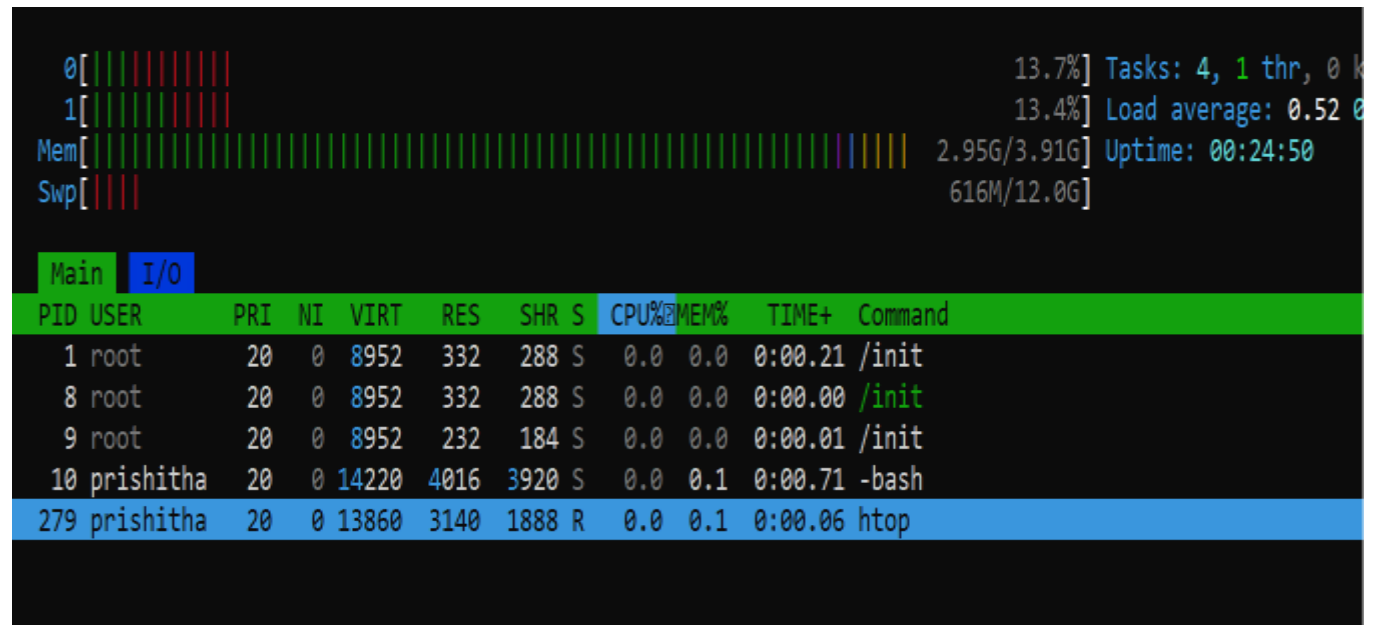
TASK [Gathering Facts] *****
ok: [localhost]

TASK [install htop package] *****
ok: [localhost]

PLAY RECAP *****
localhost                : ok=2    changed=0    unreachable=0    failed=0    skipped=0
```

htop Command Execution Showing System Monitor

```
prishitha@DELL-PC:~/devops-lab$ htop
```



Program – 8

Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins.

Aim : To configure a Jenkins CI pipeline to build a Maven project and use Ansible (CD) to deploy the generated .jar artifact to a local server.

PART A: Jenkins CI Setup for Maven Project

Step 1. Create a Maven Project

- Use mvn archetype:generate or create using IDE (like IntelliJ/Eclipse).
- Ensure it contains pom.xml and source code.
- Test locally: mvn compile test package

Step 2. Push Project to GitHub:

- Initialize Git, add remote, commit and push:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin <GitHub_Repo_URL>
git push -u origin main
```

Step 3. Install and Configure Jenkins:

- Start Jenkins and install required plugins: Maven Integration, Git, Ansible.
- Configure Maven and JDK paths in **Manage Jenkins > Global Tool Configuration**.

Step 4. Set Up Jenkins CI Job

- Open Jenkins → New Item → Select "Freestyle project".
- Under **Source Code Management**, choose Git and paste your GitHub repo URL.
- Under **Build Triggers**, enable **Poll SCM** (e.g., H/5 * * * * *).
- Under **Build** section:

- ❖ Select "Invoke top-level Maven targets"
- ❖ Goal: clean compile test package

- Add a **Windows Batch Command** to test run the jar:

```
java -cp target/your-app-1.0-SNAPSHOT.jar com.multit.App
```

- Save and **Build the job** — confirm that `.jar` is generated in `target/`.

Archive the Artifact:

- Post-build action: Select **Archive the artifacts**, and give:

`target/*.jar`

PART B: Set Up Ansible (CD) for deploying the artifact ie .JAR file generated by Jenkins

Step 1. Install Ansible in WSL (Ubuntu):

- `sudo apt update -y`
- `sudo apt install ansible -y`

Step 2. Configure Ansible Inventory and Playbook to deploy the JAR file.

- `mkdir ansible-lab`
 - `cd ansible-lab`
 - `vi inventory.ini`
- Add the following into the Inventory file

```
[local]
localhost ansible_connection=local
```

- Create the **deploy.yml** playbook:

```
---
- name: Deploy JAR
  hosts: local
  become: true

  tasks:
    - name: Copy JAR file
      copy:
        src: /mnt/c/ProgramData/Jenkins/.jenkins/workspace/your-job/target/your-app.jar
        dest: /home/your-username/ansible-lab/app.jar
        mode: '0755'

    - name: Run JAR file
      shell: nohup java -jar /home/your-username/ansible-lab/app.jar > app.log 2>&1 &
```

Step 3. Run the Playbook manually in WSL to verify the deployment:

`ansible-playbook -i inventory.ini deploy.yml`

Step 4: Automate the Deployment in Jenkins (optional)

1. Go back to the Jenkins job configuration and add a **post-build action**.
2. Add a **Windows Batch Command** to run the Ansible playbook automatically:

```
wsl ansible-playbook -i /home/your-user/ansible-lab/inventory /home/youruser/ansible-lab/deploy.yml
```

Trigger the Jenkins Build: When Jenkins builds the Maven project, it will automatically trigger the deployment using Ansible.

Step 5: Verify the Application Runs

- Check the **deployment logs** on your target machine to ensure the JAR file was deployed successfully.

Using this command : **cat app.log**

Step 6: Encounter the Manifest Error

- **During Deployment**, if you try to run the JAR and see the following error:

no main manifest attribute, in your-app-1.0-SNAPSHOT.jar

This error occurs because the `Main-Class` is missing from the `META-INF/MANIFEST.MF` file inside the JAR.

Step 7: Resolve the Manifest Error

- Fix the Manifest Error by Updating pom.xml
 - **Edit pom.xml** in the root directory of your Maven project to add the **maven-jar-plugin** configuration under the `<build>` section:

```
<build>
<plugins>
<plugin>
<artifactId>maven-jar-plugin</artifactId>
<version>3.1.0</version>
<configuration>
  <archive>
    <manifest>
      <addClasspath>true</addClasspath>
      <mainClass>com.multit.App</mainClass> <!-- Replace with main class
    </manifest>
  </archive>
</configuration>
</plugin>
</plugins>
</build>
```

Step 8: Redeploy the Artifact

- Push Changes to GitHub:

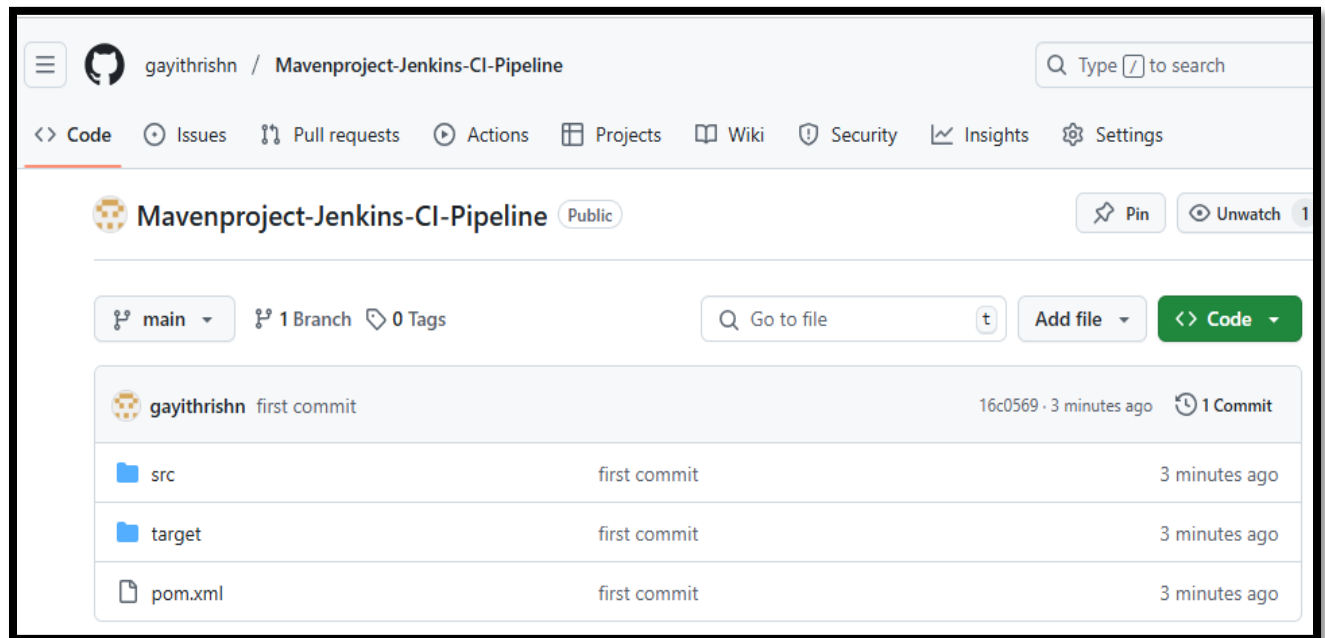
```
git add pom.xml
git commit -m "Fix manifest error by adding Main-Class"
git push -u origin main
```

Jenkins will automatically trigger the build **and deploy the new JAR file, which now includes the correct Main-Class.**

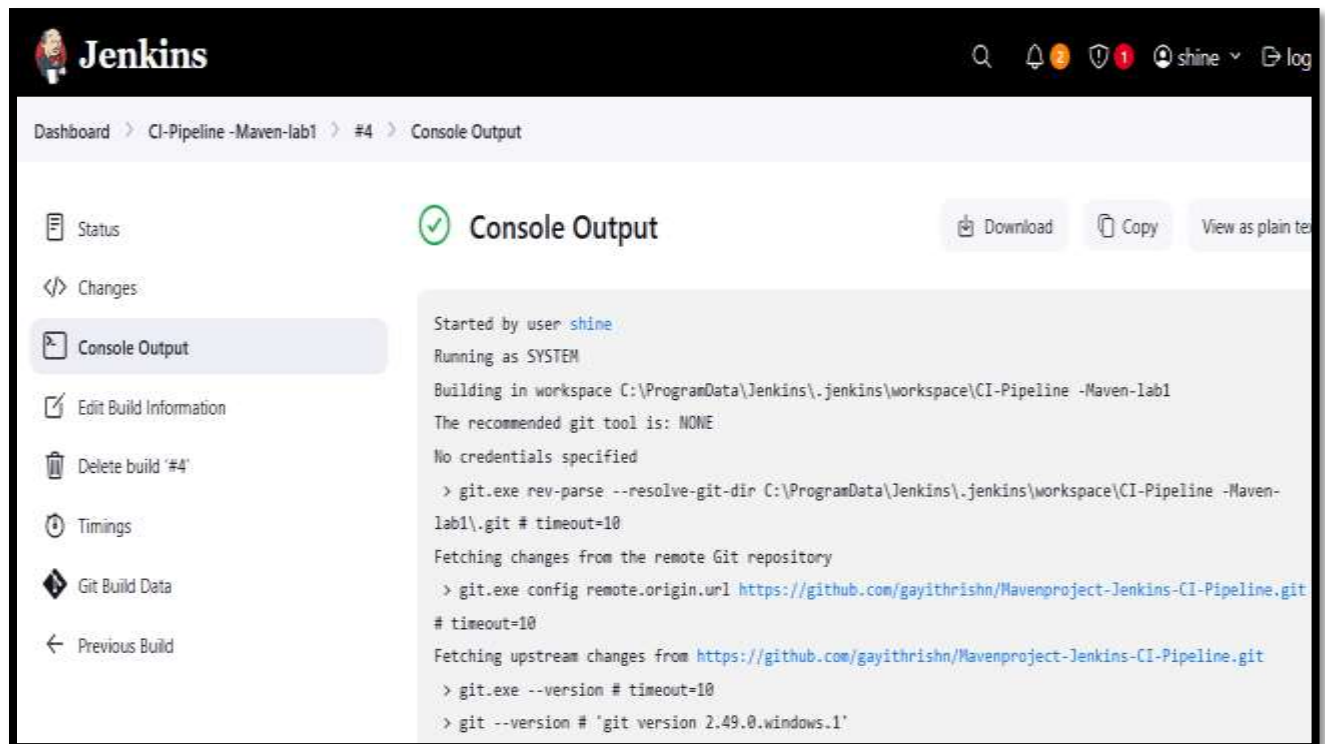
Step 9: Rerun Application

- Using this command : **cat app.log**
- now the application running without the **manifest error**.

Maven Project Hosted on GitHub



Jenkins Console Output (Build Success)



Ansible Playbook creation and Execution

```
prishitha@DELL-PC:~/Ansible-lab$ vi deploy.yml
```

```
---
- name: deploy JAR from jenkins
  hosts: local
  become: true

  tasks:
    - name: copy .jar from jenkins workspace
      copy:
        src: /mnt/c/ProgramData/Jenkins/.jenkins/workspace/CI-Pipeline -Maven-lab1/target/amazon-app-1.0-SNAPSHOT.jar
        dest: /home/prishitha/Ansible-lab/amazon-app-1.0-SNAPSHOT.jar
        mode: 0755

    - name: Run the Application
      shell: nohup java -jar /home/prishitha/Ansible-lab/amazon-app-1.0-SNAPSHOT.jar > app.log 2>&1 &
```

```
prishitha@DELL-PC: ~/Ansible-lab
prishitha@DELL-PC:~/Ansible-lab$ vi deploy.yml
prishitha@DELL-PC:~/Ansible-lab$ ansible-playbook -i inventory.ini deploy.yml

PLAY [deploy JAR from jenkins] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [copy .jar from jenkins workspace] *****
changed: [localhost]

TASK [Run the Application] *****
changed: [localhost]

PLAY RECAP *****
localhost                : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

prishitha@DELL-PC:~/Ansible-lab$
```

Application Running Confirmation (Log Output)

```
prishitha@DELL-PC:~/Ansible-lab$ cat app.log
Hello welcome to Devops lab
program executed successfully
prishitha@DELL-PC:~/Ansible-lab$
```

Manifest Attribute Error During JAR Execution

```
prishitha@DELL-PC:~/Ansible-lab$ cat app.log
no main manifest attribute, in /home/prishitha/Ansible-lab/amazon-app-1.0-SNAPSHOT.jar
prishitha@DELL-PC:~/Ansible-lab$
```

"Fixing Manifest Error: Updating pom.xml to Add Main-Class"

```
<plugin>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.0.2</version>
  <configuration>
    <archive>
      <manifest>
        <addClasspath>>true</addClasspath>
        <mainClass>com.mvit.App</mainClass>
      </manifest>
    </archive>
  </configuration>
</plugin>
```

Program - 9

Introduction to Azure DevOps: Overview of Azure DevOps Services, Setting Up an Azure DevOps Account and Project

Azure DevOps is a cloud-based suite of development tools provided by Microsoft to support the complete software development lifecycle (SDLC). It includes a set of services that helps teams plan, develop, test, and deliver applications efficiently. Azure DevOps is designed to support both continuous integration (CI) and continuous delivery (CD), and it integrates seamlessly with various development platforms.

Overview of Azure DevOps Services

Azure DevOps offers several key services, each catering to different parts of the software development lifecycle:

1. **Azure Repos** : A set of version control tools (Git or Team Foundation Version Control - TFVC) that enables you to manage your code repositories, track changes, and collaborate with your team.
2. **Azure Pipelines**: A continuous integration and continuous delivery (CI/CD) service that automates the process of building, testing, and deploying code to different environments (e.g., development, staging, production).
3. **Azure Boards**: A tool for agile project management that allows teams to plan, track, and discuss work. It includes features like Kanban boards, Scrum boards, user stories, and backlog management.
4. **Azure Test Plans**: Provides tools for manual and exploratory testing. It helps in tracking defects and managing test cases to ensure high-quality code.
5. **Azure Artifacts**: A service that enables teams to host and share packages (like NuGet, npm, and Maven) within their organization, promoting reuse and easier dependency management.
6. **Azure DevOps Services for Collaboration** : Features like dashboards, Wikis, and collaboration tools help teams work together effectively by providing visibility into the status of projects and workflows.

Setting Up Azure DevOps Account

1. Go to Azure DevOps Portal:

<https://aex.dev.azure.com>

2. Sign in with Microsoft Account

- Use your email ID to sign in
- If you don't have one, create a Microsoft account

3. Create a New Azure DevOps Organization

- On first login, it asks to **create an organization**.
- Click "**New organization**"
- Enter details:

- ❖ Organization name: Vtu-Devops-org
- ❖ Location: Choose nearest (like **South India**)

- Click **Continue**

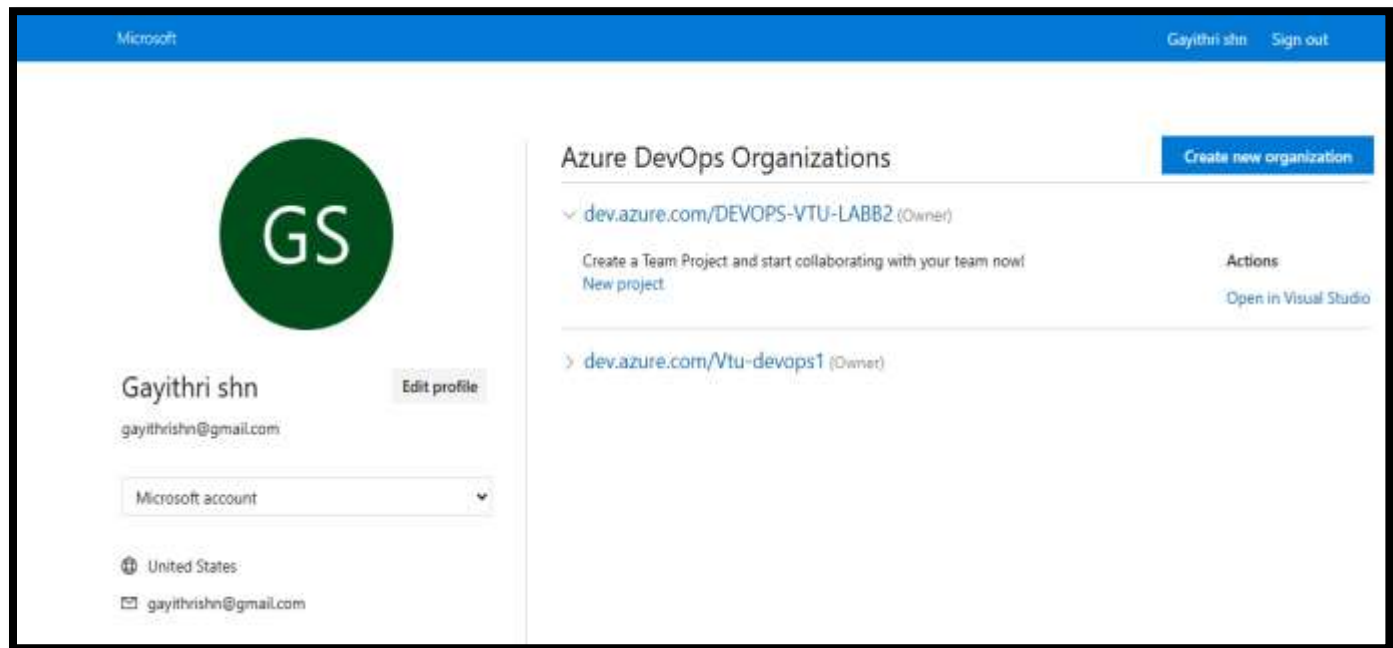
4. Create a New Azure DevOps Project

- Once organization is created, you'll see a **dashboard**.
- Click on **New project**
- Fill the details:

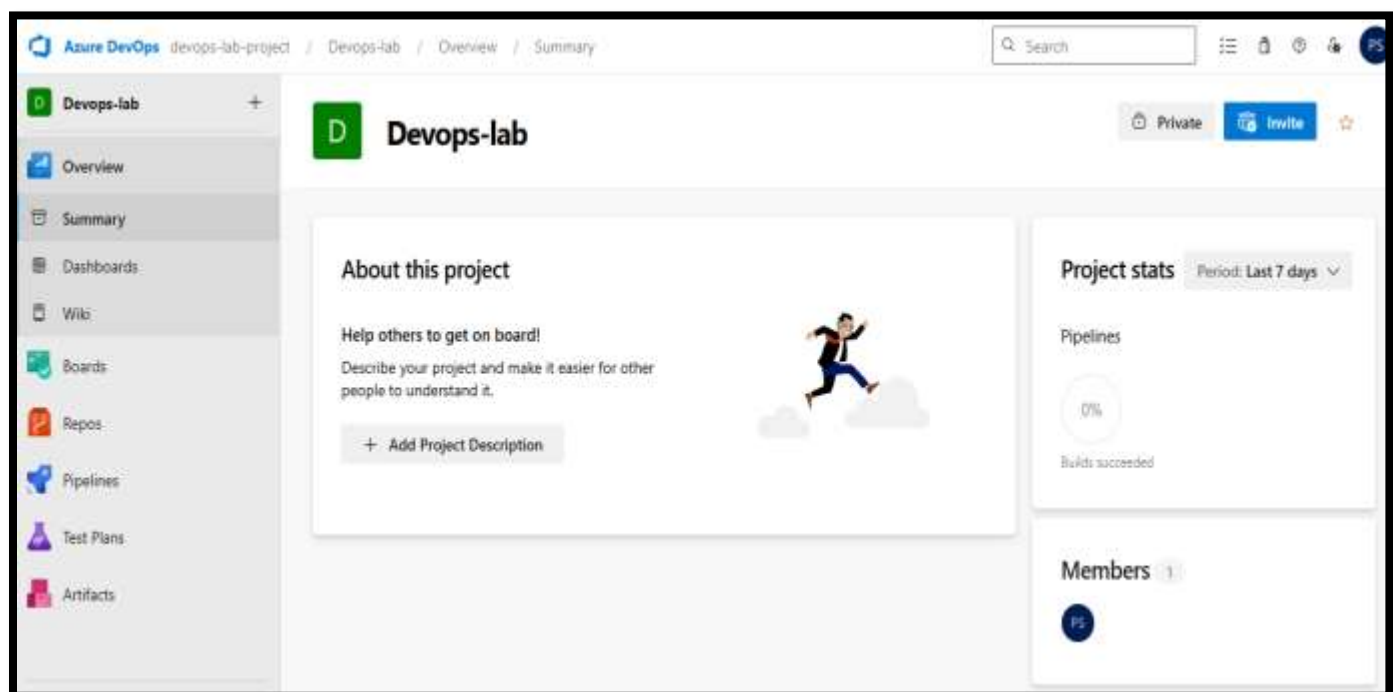
- ❖ Project name: MyFirstDevOpsProject
- ❖ Description: This is my first Azure DevOps lab project
- ❖ Visibility: Choose **Private**

- Click **Create**

Azure DevOps Account and Organization Setup



Azure DevOps Project Dashboard – Overview Page



Program - 10

Creating Build Pipelines: Building a Maven/Gradle Project with Azure Pipelines, Integrating Code Repositories (e.g., GitHub, Azure Repos), Running Unit Tests and Generating Reports

Aim: To set up a CI build pipeline for a Maven/Gradle project using Azure Pipelines with a self-hosted agent, integrate it with a code repository (e.g., GitHub), run unit tests, and generate reports.

1. Continuous Integration (CI): CI is a DevOps practice where developers frequently integrate code into a shared repository, triggering automated builds and tests to catch issues early.

2. Azure Pipelines : A cloud service that supports CI/CD to automatically build, test, and deploy code using pipelines defined in YAML or classic UI.

3. Self-Hosted Agent : Instead of using Microsoft-hosted agents, you can configure your own build machine (Windows/Linux) to run Azure DevOps pipelines. This is useful when you need specific tools, environments, or want to save cost.

4. Build Tools :

- **Maven/Gradle:** Used to build the Java project.
- **YAML:** A configuration format for defining pipeline stages, tasks, and agents.

Procedure :

Step 1: Create a Maven/Gradle Project

Step 2: Push the project to GitHub

Step 3: Create Azure DevOps Project

- Go to <https://aex.dev.azure.com>
- Click **New Project**
Project name – Lab program-10
Visibility (private/public)
- Click **Create**

Step 4: Set Up a Self-Hosted Agent

1. In Azure DevOps, go to **project Settings > Agent Pools**
2. Click on Agent Pool - > **Add Pool**
 - Name: MyLocalPool
 - Select self – hosted
 - Click create

3. Click on the created pool and click **New agent**

➤ For Download Agent

- Choose OS: Windows/Linux
- Download the ZIP file
- Extract it in a local folder (e.g., C:\azureagent)

4. Configure the Agent

- Open command prompt as Administrator
- Navigate to extracted folder `cd C:\azureagent`
- Run the following command in the extracted folder

config.cmd

Enter the following when prompted:

- ❖ Azure DevOps URL: <https://dev.azure.com/<your-org>>
- ❖ Authentication type: **PAT (Personal Access Token)**

How to create PAT

- Goto Azure devops -> Click on your **user profile** icon (top-right corner)
- In the **Personal Access Tokens** section, click "**New Token**"
- Fill in the token details:
 - **Name:** (e.g., MyAgentToken)
 - **Organization:** Select your Azure DevOps organization
 - **Expiration:** Choose duration (e.g., 30 days, 90 days)
 - **Scopes:**
 - Select **Custom defined**
 - Enable **Agent Pools → Read & Manage Build → Read and execute**
 - Click **Create**
 - Once generated, **copy the token immediately**
- ❖ Enter the PAT token you created from Azure DevOps
- ❖ Agent Pool: Select the one created above - MyLocalPool
- ❖ Agent Name: Choose a unique name (e.g Azure agent)

5. Run the Agent as service

run.cmd

Step 5 : Create a YAML Build Pipeline

- Go to **Pipelines > New Pipeline**
- Choose your repo (GitHub or Azure Repos)
- Choose Starter pipeline **YAML**

Step 6: Write the YAML Configuration

trigger:

- main

pool:

name: 'MyLocalPool' # Your self-hosted agent pool name

steps:

Step 1: Checkout the Code from GitHub

- checkout: self

displayName: 'Checkout Code from GitHub'

Step 2: Build and Run Unit Tests

- script: mvn clean test

displayName: 'Build and Run Unit Tests'

Step 3: Publish Test Results (JUnit)

- task: PublishTestResults@2

inputs:

testResultsFiles: '**/target/surefire-reports/TEST-*.xml'

testResultsFormat: 'JUnit'

failTaskOnMissingResultsFile: true

displayName: 'Publish Maven Test Results'

Step 4: Publish Build Artifacts

- task: PublishBuildArtifacts@1

inputs:

PathtoPublish: 'target'

ArtifactName: 'drop'

publishLocation: 'Container'

displayName: 'Publish Build Artifacts'

Step 7: Run Pipeline

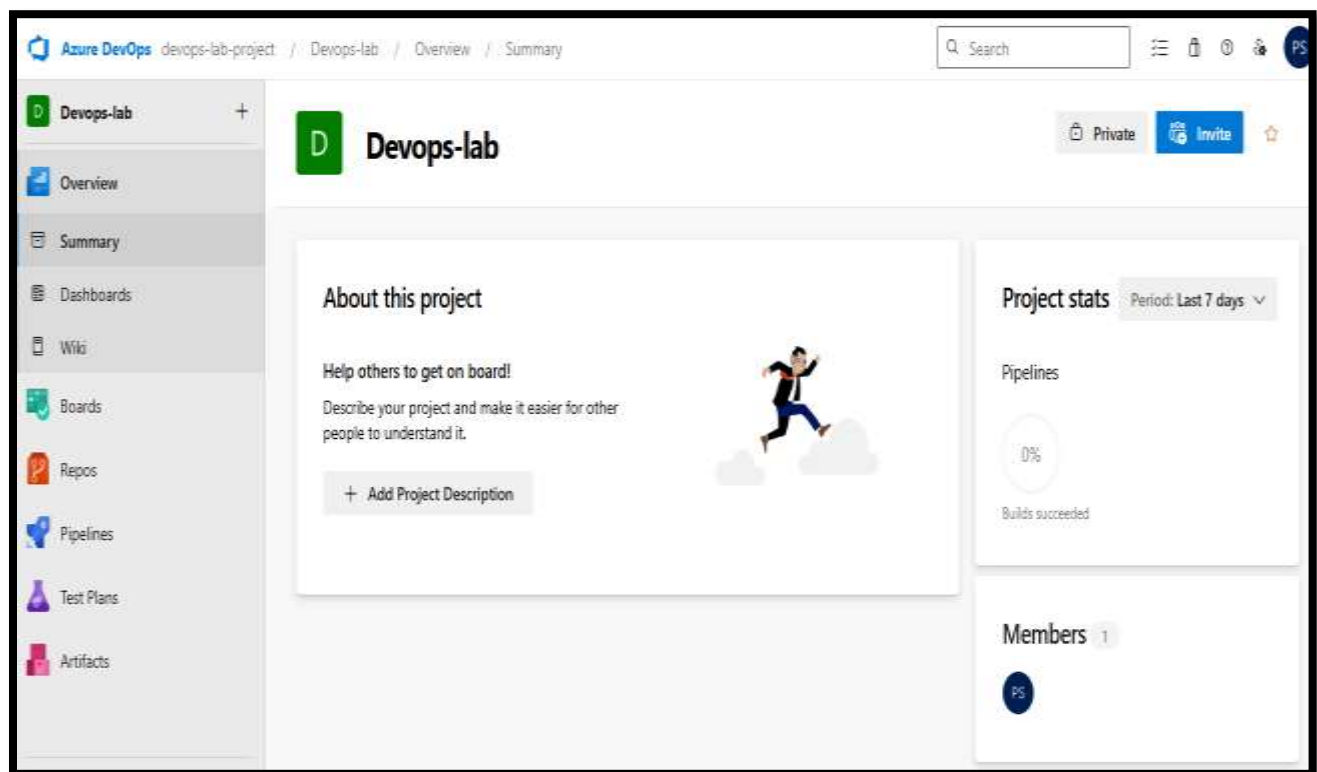
- Click **Run** to trigger the build
- Check logs for:
 - Build success/failure
 - Test results

Step 8: View Test Reports

- Go to **Pipelines > [Your Pipeline] > Runs**
- Click the latest run
- Navigate to **Tests** tab to view passed/failed unit tests

Output Screenshots

Azure DevOps Dashboard – Accessing Pipelines and Repositories



Successful Pipeline Run – Job Execution and Logs

The screenshot displays the Azure DevOps interface for a pipeline named 'gayithrishn.maven-jenkins-pipeline'. The left sidebar shows the navigation menu with 'Pipelines' selected. The main area shows the 'Jobs in run #20250514.1' with a list of jobs: 'Initialize job' (10s), 'Checkout gayithrishn/...' (34s), 'Run a one-line script' (11s), 'Run a multi-line script' (4s), 'Post-job: Checkout gayi...' (1s), and 'Finalize Job' (<1s). The 'Job' tab is selected, showing the job's execution details and logs. The logs indicate that the job was successfully completed, with a duration of 1m 2s. The logs also show the job's preparation parameters and the job's live console data, including the starting job and the execution of various commands.

Unit Test Results – All Test Cases Passed in Azure Pipeline

The screenshot displays the Azure DevOps interface for a pipeline named 'gayithrishn.PIPELINE--AZURE-DEVOPS'. The left sidebar shows the navigation menu with 'Pipelines' selected. The main area shows the 'Unit Test Results' for a pipeline run. The 'Summary' tab is selected, showing the test results. The summary indicates that 1 run(s) completed, with 1 passed and 0 failed. The test results are displayed in a table with columns for 'Total tests', 'Pass percentage', 'Run duration', and 'Tests not reported'. The 'Total tests' column shows 1 test, the 'Pass percentage' column shows 100%, the 'Run duration' column shows 384ms, and the 'Tests not reported' column shows 0. The 'Summary' section also includes a 'Run new' button and a 'View retention leases' button.

Program - 11

Creating Release Pipelines: Deploying Applications to Azure App Services, Managing Secrets and Configuration with Azure Key Vault, Hands-On: Continuous Deployment with Azure Pipelines

Aim: Creating CI/CD Pipelines using Azure DevOps with Simulated Deployment on Self-Hosted Agent (Alternative to Azure App Services)

To simulate application deployment using Azure Pipelines when Azure App Services and Classic Release Pipelines are unavailable, by using a self-hosted agent and YAML-based pipeline.

Note: Due to limitations in the Azure DevOps Free Tier (such as unavailability of Azure App Services and Classic Release Pipelines), the deployment was simulated using a local folder and a self-hosted agent. This simulation mimics the actual release pipeline behavior and fulfills the learning objective of continuous deployment using Azure Pipelines.

Tools and Technology Used:

- Azure DevOps (Free Tier)
- GitHub (for code repository)
- Maven (for build automation)
- YAML (pipeline configuration)
- Windows OS (for self-hosted agent)

Steps Followed:

1. **Create a Maven Project:**
 - Create a basic Java Maven project locally or on GitHub.
2. **Push to GitHub:**
 - Upload the project to a public GitHub repository.
3. **Create Azure DevOps Project:**
 - Sign in to Azure DevOps (<https://aex.dev.azure.com>).
 - Create a new project and link it to the GitHub repository.
4. **Configure Self-Hosted Agent:**
 - Download and configure a self-hosted agent on your Windows system.
 - Register the agent with Azure DevOps under Organization Settings > Agent Pools.

5. Create YAML Pipeline (azure-pipelines.yml):

trigger:

- main

pool:

name: MYLOCALPOOL # Name of your self-hosted agent pool

steps:

- task: Maven@3

inputs:

mavenPomFile: 'pom.xml'

goals: 'package'

- script: |

echo "Simulating deployment..."

mkdir deployed

copy target*.jar deployed\

echo "Deployment successful!"

displayName: 'Simulate Deployment'

- script: |

echo "Contents of deployed folder:"

dir deployed

displayName: 'Verify Deployment Output'

6. Run Pipeline:

- Commit the YAML file to GitHub.
- The pipeline will be triggered and executed by the self-hosted agent.

7. Verify Deployment:

1. Check the deployed folder on the self-hosted agent system.

- Open the folder where your **self-hosted agent is installed**
(e.g.C:\Users\<YourUsername>\Downloads\AZURE-AGENT-NEW1_work\1\s\
- This folder contains the source code and pipeline files during the run.

2. Look inside:

- C:\Users\<YourUsername>\Downloads\AZURE-AGENTNEW1_work\1\s\deployed
- That the `deployed` folder exists
- It contains the correct `.jar` file copied from the `target/` folder.
- This confirms that the **deployment simulation worked successfully**.

Program - 12

Practical Exercise and Wrap-Up: Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A.

To implement a complete DevOps pipeline that integrates source code management, continuous integration using Maven, and deployment using Azure Pipelines on a self-hosted agent.

Software and Tools Required:

- Azure DevOps (Free Tier Account)
- Self-Hosted Agent on Windows
- Java (JDK 17 or later)
- Apache Maven
- Git and GitHub
- Command Prompt (cmd) for Agent Execution

Step 1: Create a Simple Java Maven Project

- Create a pom.xml file for Maven with the following mainClass configuration:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <version>3.2.2</version>
      <configuration>
        <archive>
          <manifest>
            <mainClass>HelloWorld</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
```

- Initialize a Git repository and push the project to GitHub.

Step 2. Configure Azure DevOps Project

- Login to [Azure DevOps](#)
- Create a new project (e.g., DevOpsFinalProject).
- Link your GitHub repository under **Repos**.

Step 3: Set Up a Self-Hosted Agent

- Go to **Project Settings** → **Agent Pools** → **Add Pool** (e.g., MYLOCALPOOL).
- Download the agent from Azure DevOps.
- Extract it and configure with:

```
config.cmd --url https://dev.azure.com/<your-org>  
--auth pat --token <your-PAT-token>
```

- Start the agent

Step 4: Create azure-pipelines.yml

trigger:

- main

pool:

name: MYLOCALPOOL

steps:

- task: Maven@3

inputs:

mavenPomFile: 'pom.xml'

goals: 'clean package'

- script: |

echo "Simulating deployment..."

mkdir deployed

copy target*.jar deployed\

echo "Deployment successful!"

displayName: 'Simulate Deployment'

- script: |

echo "Running the JAR file..."

java -jar deployed*.jar

displayName: 'Run Application'

This YAML performs CI (build) and CD (deployment to local machine) and **runs the JAR file**, so the output will be visible in the terminal where run.cmd is running.

Step 5. Run and Verify the Pipeline

- Commit and push the YAML to GitHub
- Azure DevOps triggers the pipeline on `main` branch
- The self-hosted agent executes the pipeline:
 - Compiles and packages the Java program.
 - Copies the `.jar` to `deployed/` folder.
 - Executes the `.jar` file.

You should see the following output in the command prompt:

Hello from DevOps Final Lab!

Expected Output:

Simulating deployment...

Deployment successful!

Running the JAR file...

Hello from DevOps Final Lab!

Discussion on Best Practices in CI/CD:

1. **Use Source Control**
 - Store all code in a Git repository (GitHub/Azure Repos).
2. **Automate Builds**
 - Use Maven/Gradle for reliable, repeatable builds.
3. **Use YAML for Pipelines**
 - YAML is infrastructure-as-code and version-controlled.
4. **Keep Build and Deploy Steps Separate**
 - Use separate stages (optional) to isolate concerns.
5. **Agent Management**
 - Use self-hosted agents for controlled environments and debugging.
 - Use Microsoft-hosted agents for quick builds on clean machines.
6. **Secrets Management (Optional)**
 - Use Azure Key Vault to store passwords, API tokens securely.
7. **Monitoring and Logs**
 - Check pipeline logs and agent output for debugging